SMART TRAFFIC LIGHT CONTROL SYSTEM

Dual-Mode IoT Architecture: Synthetic Traffic Simulation & Real ESP32 Sensors

---

Project Title: Smart Traffic Light Management System
Author / Maintainer: Sparsh Goel
GitHub Profile: @gsparsh580-coder
GitHub Repository: https://github.com/gsparsh580-coder/smart-traffic-light-system
Live Web Dashboard: https://gsparsh580-coder.github.io/smart-traffic-light-system/
Communication Protocols: HiveMQ MQTT (TCP 1883 & WebSocket WSS 8884)
Hardware Controller: ESP32 DevKit V1 (HC-SR04 Ultrasonic Sensors + 12 Traffic LEDs)
Software Stack: Node-RED Dynamic Controller, Python Traffic Simulator, Tailwind HTML5 Web App
Date: October 2026

Executive Summary:
This document provides the complete source code, hardware circuit schematics, communication specifications, and deployment instructions for an intelligent traffic control system. The system dynamically computes green light timing based on queue density, provides instant priority preemption for emergency vehicles, and operates in dual data ingestion modes: synthetic Poisson arrivals and physical ESP32 ultrasonic telemetry.

1. System Architecture & MQTT Topic Schema

The system decouples edge hardware and web supervisors through HiveMQ MQTT broker. Either the Python simulation generator or physical ESP32 publishes vehicle densities. Node-RED ingests this stream, calculates adaptive green durations, and publishes actuation commands back to the lights.

Topic	Payload Type	Function
smart_traffic/demo/system/mode	JSON String	Toggles SYNTHETIC vs REAL_ESP32 data mode
smart_traffic/demo/lane1/density	JSON Object	Northbound queue size & ultrasonic distance (cm)
smart_traffic/demo/lane2/density	JSON Object	Eastbound queue size & ultrasonic distance (cm)
smart_traffic/demo/lane3/density	JSON Object	Southbound queue size & ultrasonic distance (cm)
smart_traffic/demo/lane4/density	JSON Object	Westbound queue size & ultrasonic distance (cm)
smart_traffic/demo/control/lights	JSON Object	Master 4-way active states, countdown, and phases
smart_traffic/demo/lane[1-4]/light	String (Enum)	Discrete GPIO light commands (RED, YELLOW, GREEN)
smart_traffic/demo/control/emergency	JSON Object	Emergency vehicle preemption trigger (OVERRIDE / CLEAR)
smart_traffic/demo/control/manual	JSON Object	Operator override commands (FORCE_LANE, ALL_RED, AUTO)

2. ESP32 Hardware Pinouts & Circuit Configuration

Intersection Lane	Ultrasonic Sensor (HC-SR04)	Red LED	Yellow LED	Green LED
Lane 1 (North)	Trig: GPIO 5, Echo: GPIO 18	GPIO 13	GPIO 12	GPIO 14
Lane 2 (East)	Trig: GPIO 19, Echo: GPIO 21	GPIO 27	GPIO 33	GPIO 32
Lane 3 (South)	Trig: GPIO 22, Echo: GPIO 23	GPIO 15	GPIO 2	GPIO 4
Lane 4 (West)	Trig: GPIO 25, Echo: GPIO 26	GPIO 16	GPIO 17	GPIO 5
Emergency Override	Push Button: GPIO 34 (INPUT_PULLUP)	--	--	--
Heartbeat / Status	Onboard Blue LED: GPIO 2	--	--	--

Code Listing: 1. ESP32 Arduino Firmware

Path: esp32_firmware/esp32_smart_traffic.ino | **Type:** C++ / Arduino Sketch

```
/**
 * ================================================================================
 * Smart Traffic Light Control System - ESP32 Firmware
 * ================================================================================
 * Hardware: ESP32 DevKit V1
 * Sensors: HC-SR04 Ultrasonic Sensors (or IR proximity sensors)
 * Actuators: 4x Traffic Light LEDs (Red, Yellow, Green)
 * Connectivity: WiFi + HiveMQ MQTT Broker
 *
 * Capabilities:
 *  - Real-time vehicle density detection (Distance to vehicle queue)
 *  - HiveMQ MQTT telemetry streaming (lane density & emergency trigger)
 *  - Remote actuation from Node-RED / Web Dashboard
 *  - Offline Autonomous Safe Failsafe if network drops
 * ================================================================================
 */

#include <WiFi.h>
#include <PubSubClient.h>

// Fallback configuration if secrets.h is not present
#if __has_include("secrets.h")
  #include "secrets.h"
#else
  const char* WIFI_SSID         = "YOUR_WIFI_SSID";
  const char* WIFI_PASSWORD     = "YOUR_WIFI_PASSWORD";
  const char* MQTT_BROKER       = "broker.hivemq.com";
  const int   MQTT_PORT         = 1883;
  const char* MQTT_USER         = "";
  const char* MQTT_PASSWORD     = "";
  const char* MQTT_TOPIC_PREFIX = "smart_traffic/demo";
#endif

// ================================================================================
// HARDWARE PIN DEFINITIONS (ESP32 DevKit V1)
// ================================================================================

// Number of controlled intersection lanes (2 for compact demo, 4 for full 4-way)
#define NUM_LANES 4

// Ultrasonic Sensor Pins (Trig & Echo)
const int TRIG_PINS[NUM_LANES] = { 5, 19, 22, 25 };
const int ECHO_PINS[NUM_LANES] = { 18, 21, 23, 26 };

// Traffic Light LED Pins (Red, Yellow, Green for each Lane)
const int LED_RED[NUM_LANES]    = { 13, 27, 15, 16 };
const int LED_YELLOW[NUM_LANES] = { 12, 33,  2, 17 };
const int LED_GREEN[NUM_LANES]  = { 14, 32,  4,  5 };

// Emergency / Pedestrian Push Button Pin (Internal Pullup)
const int EMERGENCY_BUTTON_PIN = 34;

// Onboard Status LED (Blinks on MQTT activity)
const int STATUS_LED = 2;

// ================================================================================
// STATE VARIABLES & OBJECTS
// ================================================================================
WiFiClient espClient;
PubSubClient mqttClient(espClient);

// Lane States
```

```
enum LightState { LIGHT_RED, LIGHT_YELLOW, LIGHT_GREEN };
LightState laneLights[NUM_LANES] = { LIGHT_GREEN, LIGHT_RED, LIGHT_RED, LIGHT_RED };

int laneDistances[NUM_LANES] = { 100, 100, 100, 100 }; // in cm
int laneVehicles[NUM_LANES]  = { 0, 0, 0, 0 };

unsigned long lastSensorPublishTime = 0;
const unsigned long SENSOR_PUBLISH_INTERVAL = 1500; // ms

unsigned long lastEmergencyDebounce = 0;
bool emergencyActive = false;

// Offline Autonomous Fallback Timing
bool offlineMode = false;
unsigned long lastOfflineCycleTime = 0;
int offlineActiveLane = 0;
int offlinePhase = 0; // 0=Green, 1=Yellow

// ============================================================================
// HELPER: ULTRASONIC SENSOR MEASUREMENT
// ============================================================================
int measureDistanceCm(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Measure pulse duration (timeout 25ms ~ 4.3 meters max)
  long duration = pulseIn(echoPin, HIGH, 25000);
  if (duration <= 0) {
    return 150; // No obstacle / empty road default
  }

  int distanceCm = duration * 0.034 / 2;
  return constrain(distanceCm, 2, 150);
}

// Convert distance in cm to estimated vehicle queue count
// Closer vehicles = shorter distance = higher density
int distanceToVehicleCount(int distanceCm) {
  if (distanceCm < 10) return 18; // Heavy queue right at stop line
  if (distanceCm < 25) return 12;
  if (distanceCm < 45) return 7;
  if (distanceCm < 70) return 3;
  if (distanceCm < 100) return 1;
  return 0; // Clear road
}

// ============================================================================
// HARDWARE ACTUATION: LED CONTROL
// ============================================================================
void updateLaneLED(int laneIdx, LightState state) {
  laneLights[laneIdx] = state;

  digitalWrite(LED_RED[laneIdx],    state == LIGHT_RED    ? HIGH : LOW);
  digitalWrite(LED_YELLOW[laneIdx], state == LIGHT_YELLOW ? HIGH : LOW);
  digitalWrite(LED_GREEN[laneIdx],  state == LIGHT_GREEN  ? HIGH : LOW);
}

void applyLightStateString(int laneIdx, const String& stateStr) {
  if (laneIdx < 0 || laneIdx >= NUM_LANES) return;
```

```
  if (stateStr.equalsIgnoreCase("GREEN")) {
    updateLaneLED(laneIdx, LIGHT_GREEN);
  } else if (stateStr.equalsIgnoreCase("YELLOW")) {
    updateLaneLED(laneIdx, LIGHT_YELLOW);
  } else {
    updateLaneLED(laneIdx, LIGHT_RED);
  }
}

// ============================================================================
// MQTT CALLBACK: REMOTE CONTROLLER COMMANDS
// ============================================================================
void onMqttMessage(char* topic, byte* payload, unsigned int length) {
  String message = "";
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }
  message.trim();

  Serial.printf("[MQTT RX] %s -> %s\n", topic, message.c_str());

  // Individual lane light topics: <prefix>/lane1/light
  for (int i = 0; i < NUM_LANES; i++) {
    char expectedTopic[80];
    snprintf(expectedTopic, sizeof(expectedTopic), "%s/lane%d/light", MQTT_TOPIC_PREFIX, i + 1);
    if (strcmp(topic, expectedTopic) == 0) {
      applyLightStateString(i, message);
      return;
    }
  }

  // Master light control JSON topic: <prefix>/control/lights
  char masterTopic[80];
  snprintf(masterTopic, sizeof(masterTopic), "%s/control/lights", MQTT_TOPIC_PREFIX);
  if (strcmp(topic, masterTopic) == 0) {
    // Quick parse for light states in JSON without heavy library dependencies
    for (int i = 0; i < NUM_LANES; i++) {
      char key[20];
      snprintf(key, sizeof(key), "\"lane%d\":\"", i + 1);
      int idx = message.indexOf(key);
      if (idx != -1) {
        idx += strlen(key);
        int endQuote = message.indexOf("\"", idx);
        if (endQuote != -1) {
          String val = message.substring(idx, endQuote);
          applyLightStateString(i, val);
        }
      }
    }
  }

  // Emergency override notification: <prefix>/control/emergency
  char emergTopic[80];
  snprintf(emergTopic, sizeof(emergTopic), "%s/control/emergency", MQTT_TOPIC_PREFIX);
  if (strcmp(topic, emergTopic) == 0) {
    if (message.indexOf("OVERRIDE") != -1) {
      emergencyActive = true;
      Serial.println("[!] Emergency Override Active from MQTT!");
    } else if (message.indexOf("CLEAR") != -1) {
      emergencyActive = false;
      Serial.println("[✓] Emergency Cleared.");
```

```
    }
  }
}

// ================================================================================
// WIFI & MQTT CONNECTION MANAGEMENT
// ================================================================================
void setupWiFi() {
  Serial.printf("Connecting to WiFi: %s", WIFI_SSID);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 25) {
    delay(400);
    Serial.print(".");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n[✓] WiFi Connected!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("\n[✗] WiFi Connection Failed. Entering Autonomous Offline Mode.");
    offlineMode = true;
  }
}

void reconnectMqtt() {
  if (WiFi.status() != WL_CONNECTED) return;

  while (!mqttClient.connected()) {
    Serial.printf("Connecting to HiveMQ (%s:%d)... ", MQTT_BROKER, MQTT_PORT);
    String clientId = "ESP32_TrafficNode_" + String(random(0xffff), HEX);

    bool connected = false;
    if (strlen(MQTT_USER) > 0) {
      connected = mqttClient.connect(clientId.c_str(), MQTT_USER, MQTT_PASSWORD);
    } else {
      connected = mqttClient.connect(clientId.c_str());
    }

    if (connected) {
      Serial.println("[✓] HiveMQ Connected!");
      offlineMode = false;

      // Publish Device Online Announcement
      char statusTopic[80];
      snprintf(statusTopic, sizeof(statusTopic), "%s/system/status", MQTT_TOPIC_PREFIX);
      mqttClient.publish(statusTopic, "{\"device\":\"ESP32\",\"status\":\"ONLINE\",\"mode\":\"REAL_ESP32\"}", true);

      // Subscribe to control topics
      char subPattern[80];
      snprintf(subPattern, sizeof(subPattern), "%s/control/#", MQTT_TOPIC_PREFIX);
      mqttClient.subscribe(subPattern);

      for (int i = 0; i < NUM_LANES; i++) {
        char lightSub[80];
        snprintf(lightSub, sizeof(lightSub), "%s/lane%d/light", MQTT_TOPIC_PREFIX, i + 1);
        mqttClient.subscribe(lightSub);
      }
```

```
    } else {
      Serial.printf("[✗] MQTT failed (rc=%d). Retrying in 4s...\n", mqttClient.state());
      delay(4000);
      break; // Let main loop run to keep offline lights cycling
    }
  }
}

// ============================================================================
// OFFLINE AUTONOMOUS SAFE CYCLE
// ============================================================================
void runOfflineFailsafeCycle() {
  unsigned long now = millis();
  unsigned long elapsed = now - lastOfflineCycleTime;

  if (offlinePhase == 0) {
    // Green Phase (10 seconds)
    if (elapsed >= 10000) {
      offlinePhase = 1;
      lastOfflineCycleTime = now;
      updateLaneLED(offlineActiveLane, LIGHT_YELLOW);
    }
  } else if (offlinePhase == 1) {
    // Yellow Phase (3 seconds) -> Transition to next lane
    if (elapsed >= 3000) {
      updateLaneLED(offlineActiveLane, LIGHT_RED);
      offlineActiveLane = (offlineActiveLane + 1) % NUM_LANES;
      offlinePhase = 0;
      lastOfflineCycleTime = now;
      updateLaneLED(offlineActiveLane, LIGHT_GREEN);
    }
  }
}

// ============================================================================
// ARDUINO SETUP & LOOP
// ============================================================================
void setup() {
  Serial.begin(115200);
  delay(500);
  Serial.println("\n================================================");
  Serial.println("  ESP32 Smart Traffic Light Controller Start  ");
  Serial.println("================================================");

  // Initialize Pins
  pinMode(STATUS_LED, OUTPUT);
  pinMode(EMERGENCY_BUTTON_PIN, INPUT_PULLUP);

  for (int i = 0; i < NUM_LANES; i++) {
    pinMode(TRIG_PINS[i], OUTPUT);
    pinMode(ECHO_PINS[i], INPUT);
    digitalWrite(TRIG_PINS[i], LOW);

    pinMode(LED_RED[i], OUTPUT);
    pinMode(LED_YELLOW[i], OUTPUT);
    pinMode(LED_GREEN[i], OUTPUT);

    // Initial safe state: Lane 1 Green, other lanes Red
    updateLaneLED(i, (i == 0) ? LIGHT_GREEN : LIGHT_RED);
  }

  // Initialize Network & MQTT
```

```
  setupWiFi();
  mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
  mqttClient.setCallback(onMqttMessage);
  mqttClient.setBufferSize(512);

  lastOfflineCycleTime = millis();
}

void loop() {
  // Check MQTT Connection
  if (!mqttClient.connected()) {
    reconnectMqtt();
    if (!mqttClient.connected()) {
      runOfflineFailsafeCycle();
    }
  } else {
    mqttClient.loop();
  }

  // 1. Read Emergency Push Button
  if (digitalRead(EMERGENCY_BUTTON_PIN) == LOW) {
    if (millis() - lastEmergencyDebounce > 1000) {
      lastEmergencyDebounce = millis();
      emergencyActive = !emergencyActive;
      Serial.printf("[BUTTON] Emergency Trigger Toggled: %s\n", emergencyActive ? "ACTIVE" : "CLEARED");

      char emergTopic[80];
      snprintf(emergTopic, sizeof(emergTopic), "%s/control/emergency", MQTT_TOPIC_PREFIX);
      char payload[100];
      snprintf(payload, sizeof(payload), "{\"action\":\"%s\",\"lane\":1,\"source\":\"ESP32_BUTTON\"}",
               emergencyActive ? "OVERRIDE" : "CLEAR");
      mqttClient.publish(emergTopic, payload);
    }
  }

  // 2. Periodic Sensor Polling & HiveMQ Telemetry Publishing
  unsigned long now = millis();
  if (now - lastSensorPublishTime >= SENSOR_PUBLISH_INTERVAL) {
    lastSensorPublishTime = now;

    for (int i = 0; i < NUM_LANES; i++) {
      int dist = measureDistanceCm(TRIG_PINS[i], ECHO_PINS[i]);
      laneDistances[i] = dist;
      laneVehicles[i]  = distanceToVehicleCount(dist);

      if (mqttClient.connected()) {
        char topic[80];
        snprintf(topic, sizeof(topic), "%s/lane%d/density", MQTT_TOPIC_PREFIX, i + 1);

        char payload[140];
        snprintf(payload, sizeof(payload),
                 "{\"lane\":%d,\"distance_cm\":%d,\"vehicle_count\":%d,\"source\":\"ESP32_REAL\"}",
                 i + 1, dist, laneVehicles[i]);

        mqttClient.publish(topic, payload);
      }
    }

    // Toggle Status LED pulse
    digitalWrite(STATUS_LED, !digitalRead(STATUS_LED));
  }
}
```

Code Listing: 2. ESP32 Secrets Configuration Template

Path: esp32_firmware/secrets.h.example | **Type:** C++ Header

```
/**
 * Smart Traffic Light Control System - Configuration Credentials
 * Rename this file to 'secrets.h' and fill in your WiFi and HiveMQ credentials.
 */

#ifndef SECRETS_H
#define SECRETS_H

// WiFi Configuration
const char* WIFI_SSID     = "YOUR_WIFI_SSID";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// HiveMQ Broker Configuration
// Option 1: HiveMQ Public Free Broker (No password needed)
const char* MQTT_BROKER   = "broker.hivemq.com";
const int   MQTT_PORT     = 1883;
const char* MQTT_USER     = "";
const char* MQTT_PASSWORD = "";

// Option 2: HiveMQ Cloud (Private Cluster with TLS)
// const char* MQTT_BROKER   = "your-cluster-id.s1.eu.hivemq.cloud";
// const int   MQTT_PORT     = 8883; // Requires WiFiClientSecure
// const char* MQTT_USER     = "hivemq_username";
// const char* MQTT_PASSWORD = "hivemq_password";

// Base MQTT Topic (Keep matching between ESP32, Node-RED, and Web Dashboard)
const char* MQTT_TOPIC_PREFIX = "smart_traffic/demo";

#endif
```

Code Listing: 3. Python Synthetic Traffic Simulator

Path: simulator/traffic_simulator.py | **Type:** Python 3

```
#!/usr/bin/env python3
"""
Smart Traffic Light Control System - Synthetic Traffic Data Generator
Simulates realistic 4-way intersection vehicle queues, arrival rates,
and emergency vehicle events, publishing live telemetry to HiveMQ MQTT.
"""

import sys
import time
import json
import random
import argparse
import threading
from datetime import datetime

try:
    import paho.mqtt.client as mqtt
    from paho.mqtt.enums import CallbackAPIVersion
    PAHO_V2 = True
except (ImportError, AttributeError):
    try:
        import paho.mqtt.client as mqtt
        PAHO_V2 = False
    except ImportError:
        print("[!] paho-mqtt is required. Run: pip install paho-mqtt")
        sys.exit(1)

# Default Configuration
DEFAULT_BROKER = "broker.hivemq.com"
DEFAULT_PORT = 1883
DEFAULT_TOPIC_PREFIX = "smart_traffic/demo"

# Terminal Color Codes
CYAN = "\033[96m"
GREEN = "\033[92m"
YELLOW = "\033[93m"
RED = "\033[91m"
MAGENTA = "\033[95m"
BOLD = "\033[1m"
RESET = "\033[0m"


class TrafficSimulator:
    def __init__(self, broker, port, prefix, interval=2.0):
        self.broker = broker
        self.port = port
        self.prefix = prefix.rstrip("/")
        self.interval = interval
        self.running = False

        # Lane States: vehicle count (0 to 30)
        self.lanes = {
            1: {"name": "Northbound", "vehicles": 5, "distance_cm": 35, "light": "RED"},
            2: {"name": "Eastbound",  "vehicles": 3, "distance_cm": 50, "light": "RED"},
            3: {"name": "Southbound", "vehicles": 8, "distance_cm": 20, "light": "RED"},
            4: {"name": "Westbound",  "vehicles": 4, "distance_cm": 42, "light": "RED"},
        }

        # Simulation Mode: AUTO, RUSH_HOUR, BALANCED, MANUAL
        self.sim_mode = "AUTO"
        self.emergency_lane = None
        self.active_system_mode = "SYNTHETIC"  # or REAL_ESP32
```

```
        self.light_cycle_info = {"active_lane": 1, "remaining_sec": 0}

        # Setup MQTT Client
        client_id = f"TrafficSimulator_{random.randint(1000, 9999)}"
        if PAHO_V2:
            self.client = mqtt.Client(CallbackAPIVersion.VERSION2, client_id=client_id)
        else:
            self.client = mqtt.Client(client_id=client_id)

        self.client.on_connect = self._on_connect
        self.client.on_message = self._on_message
        self.client.on_disconnect = self._on_disconnect

    def _on_connect(self, client, userdata, flags, rc, properties=None):
        if rc == 0:
            print(f"{GREEN}[✓] Connected to HiveMQ Broker ({self.broker}:{self.port}){RESET}")
            # Subscribe to traffic light state and system mode updates
            client.subscribe(f"{self.prefix}/control/lights")
            client.subscribe(f"{self.prefix}/system/mode")
            client.subscribe(f"{self.prefix}/control/emergency")

            # Announce synthetic mode is active
            self.publish_system_mode()
        else:
            print(f"{RED}[✗] Connection failed with code {rc}{RESET}")

    def _on_disconnect(self, client, userdata, rc, properties=None):
        print(f"{YELLOW}[!] Disconnected from broker (rc={rc}){RESET}")

    def _on_message(self, client, userdata, msg):
        topic = msg.topic
        payload_str = msg.payload.decode("utf-8", errors="ignore")
        try:
            data = json.loads(payload_str)
        except Exception:
            data = payload_str

        # Update lights feedback from Node-RED or controller
        if topic == f"{self.prefix}/control/lights":
            if isinstance(data, dict):
                for lane_id in (1, 2, 3, 4):
                    lane_key = f"lane{lane_id}"
                    if lane_key in data:
                        self.lanes[lane_id]["light"] = data[lane_key]
                self.light_cycle_info["active_lane"] = data.get("active_lane", 1)
                self.light_cycle_info["remaining_sec"] = data.get("remaining_sec", 0)

        elif topic == f"{self.prefix}/system/mode":
            if isinstance(data, dict):
                self.active_system_mode = data.get("mode", self.active_system_mode)
            else:
                self.active_system_mode = str(data)

        elif topic == f"{self.prefix}/control/emergency":
            if isinstance(data, dict):
                if data.get("action") == "OVERRIDE":
                    self.emergency_lane = data.get("lane")
                elif data.get("action") == "CLEAR":
                    self.emergency_lane = None

    def publish_system_mode(self):
        payload = {
```

```
            "mode": "SYNTHETIC",
            "source": "Python Traffic Simulator",
            "timestamp": int(time.time()),
        }
        self.client.publish(f"{self.prefix}/system/mode", json.dumps(payload), retain=True)

    def publish_lane_density(self, lane_id):
        lane = self.lanes[lane_id]
        # Invert distance for sensor emulation: high vehicles = short ultrasonic distance (cm)
        # 0 vehicles ~ 100cm, 20 vehicles ~ 10cm
        sensor_dist = max(5, int(100 - (lane["vehicles"] * 4.5)))
        lane["distance_cm"] = sensor_dist

        payload = {
            "lane": lane_id,
            "lane_name": lane["name"],
            "vehicle_count": lane["vehicles"],
            "queue_length_m": lane["vehicles"] * 5.0,  # avg 5m per vehicle
            "distance_cm": sensor_dist,
            "is_emergency": (self.emergency_lane == lane_id),
            "timestamp": int(time.time()),
        }
        topic = f"{self.prefix}/lane{lane_id}/density"
        self.client.publish(topic, json.dumps(payload))

    def trigger_emergency(self, lane_id):
        self.emergency_lane = lane_id
        # Boost lane vehicles to represent congestion behind emergency vehicle
        self.lanes[lane_id]["vehicles"] = max(self.lanes[lane_id]["vehicles"], 12)
        payload = {
            "action": "OVERRIDE",
            "lane": lane_id,
            "vehicle_type": "AMBULANCE",
            "timestamp": int(time.time())
        }
        self.client.publish(f"{self.prefix}/control/emergency", json.dumps(payload))
        print(f"\n{RED}{BOLD}[■ EMERGENCY VEHICLE TRIGGERED ON LANE {lane_id} ({self.lanes[lane_id]['name']})!]{RESET}\n")

    def clear_emergency(self):
        self.emergency_lane = None
        payload = {
            "action": "CLEAR",
            "lane": 0,
            "timestamp": int(time.time())
        }
        self.client.publish(f"{self.prefix}/control/emergency", json.dumps(payload))
        print(f"\n{GREEN}[✓ Emergency status cleared]{RESET}\n")

    def update_simulation_physics(self):
        """Simulate vehicle arrivals and departures based on traffic light states"""
        for lane_id, lane in self.lanes.items():
            is_green = (lane["light"] == "GREEN")

            # 1. Vehicles depart if light is GREEN
            if is_green and lane["vehicles"] > 0:
                departure_rate = random.randint(1, 3)
                lane["vehicles"] = max(0, lane["vehicles"] - departure_rate)

            # 2. Vehicle arrivals depending on simulation profile
            if self.sim_mode == "RUSH_HOUR":
                # Heavy traffic on Lanes 1 & 3 (North-South arterial road)
                if lane_id in (1, 3):
```

```
                    arrivals = random.choices([1, 2, 3, 4], weights=[0.2, 0.4, 0.3, 0.1])[0]
                else:
                    arrivals = random.choices([0, 1, 2], weights=[0.6, 0.3, 0.1])[0]
            elif self.sim_mode == "BALANCED":
                arrivals = random.choices([0, 1, 2], weights=[0.4, 0.4, 0.2])[0]
            else:  # AUTO with natural random waves
                arrivals = random.choices([0, 1, 2, 3], weights=[0.35, 0.4, 0.2, 0.05])[0]

            # Clamp max queue to 30
            lane["vehicles"] = min(30, lane["vehicles"] + arrivals)

    def print_terminal_dashboard(self):
        # Clear line and print compact status board
        sys.stdout.write("\033[H\033[J")  # ANSI clear screen
        print(f"{CYAN}{BOLD}================================================================================{RESET}")
        print(f"{CYAN}{BOLD}       SMART TRAFFIC LIGHT SYSTEM - SYNTHETIC DATA SIMULATOR           {RESET}")
        print(f"{CYAN}{BOLD}================================================================================{RESET}")
        print(f"Broker: {self.broker}:{self.port} | Topic Prefix: {self.prefix}")
        print(f"System Mode: {MAGENTA}{self.active_system_mode}{RESET} | Simulation Profile: {YELLOW}{self.sim_mode}{RESET}")
        if self.emergency_lane:
            print(f"Emergency Status: {RED}{BOLD}ACTIVE ON LANE {self.emergency_lane} ({self.lanes[self.emergency_lane]['name']}
        else:
            print(f"Emergency Status: {GREEN}NORMAL{RESET}")
        print("-" * 72)

        # Print 4 lanes
        for lane_id, data in self.lanes.items():
            light_color = RED if data["light"] == "RED" else (YELLOW if data["light"] == "YELLOW" else GREEN)
            light_badge = f"{light_color}[ {data['light']:^6} ]{RESET}"
            bar_len = min(25, data["vehicles"])
            queue_bar = f"{CYAN}{'■' * bar_len}{'■' * (25 - bar_len)}{RESET}"
            emerg_tag = f" {RED}{BOLD}[EMERGENCY]{RESET}" if self.emergency_lane == lane_id else ""

            print(f"Lane {lane_id} ({data['name']:<11}): {light_badge}  Queue: {queue_bar} ({data['vehicles']:2d} cars, {data['

        print("-" * 72)
        print(f"Controls: [1-4] Emergency on Lane | [C] Clear Emergency | [R] Rush Hour")
        print(f"          [B] Balanced Mode     | [A] Auto Mode        | [Q] Quit")
        print(f"Updated: {datetime.now().strftime('%H:%M:%S')} (publishing every {self.interval}s)")

    def run(self):
        self.running = True
        print(f"Connecting to {self.broker}...")
        try:
            self.client.connect(self.broker, self.port, 60)
            self.client.loop_start()
        except Exception as e:
            print(f"{RED}Failed to connect: {e}{RESET}")
            return

        # Start input listener thread
        input_thread = threading.Thread(target=self._keyboard_listener, daemon=True)
        input_thread.start()

        try:
            while self.running:
                # Update simulation physics
                self.update_simulation_physics()

                # Publish data for all 4 lanes
                for lane_id in range(1, 5):
                    self.publish_lane_density(lane_id)
```

```
                # Render dashboard to terminal
                self.print_terminal_dashboard()

                time.sleep(self.interval)
        except KeyboardInterrupt:
            print("\nShutting down simulator...")
        finally:
            self.client.loop_stop()
            self.client.disconnect()
            print("Simulator stopped.")

    def _keyboard_listener(self):
        """Simple interactive CLI listener without external key-listener dependencies"""
        while self.running:
            try:
                line = sys.stdin.readline().strip().lower()
                if not line:
                    continue
                cmd = line[0]
                if cmd in ("1", "2", "3", "4"):
                    self.trigger_emergency(int(cmd))
                elif cmd == "c":
                    self.clear_emergency()
                elif cmd == "r":
                    self.sim_mode = "RUSH_HOUR"
                    print(f"\n{YELLOW}[Mode switched to RUSH_HOUR: High North-South Traffic]{RESET}")
                elif cmd == "b":
                    self.sim_mode = "BALANCED"
                    print(f"\n{GREEN}[Mode switched to BALANCED Traffic]{RESET}")
                elif cmd == "a":
                    self.sim_mode = "AUTO"
                    print(f"\n{CYAN}[Mode switched to AUTO Dynamic Traffic]{RESET}")
                elif cmd == "q":
                    self.running = False
                    break
            except Exception:
                break


def main():
    parser = argparse.ArgumentParser(description="Smart Traffic Light Synthetic Data Simulator")
    parser.add_argument("--broker", default=DEFAULT_BROKER, help=f"MQTT broker hostname (default: {DEFAULT_BROKER})")
    parser.add_argument("--port", type=int, default=DEFAULT_PORT, help=f"MQTT broker port (default: {DEFAULT_PORT})")
    parser.add_argument("--prefix", default=DEFAULT_TOPIC_PREFIX, help=f"MQTT topic prefix (default: {DEFAULT_TOPIC_PREFIX})")
    parser.add_argument("--interval", type=float, default=2.0, help="Publishing interval in seconds (default: 2.0)")

    args = parser.parse_args()
    sim = TrafficSimulator(broker=args.broker, port=args.port, prefix=args.prefix, interval=args.interval)
    sim.run()


if __name__ == "__main__":
    main()
```

Code Listing: 4. Traffic Simulation Presets

Path: simulator/scenarios.json | **Type:** JSON Dataset

```
{
  "scenarios": [
    {
      "id": "morning_rush",
      "name": "Morning Commute Rush",
      "description": "Heavy inbound traffic on Northbound (Lane 1) and Southbound (Lane 3).",
      "lanes": {
        "lane1": { "vehicle_count": 22, "distance_cm": 10 },
        "lane2": { "vehicle_count": 4,  "distance_cm": 75 },
        "lane3": { "vehicle_count": 18, "distance_cm": 15 },
        "lane4": { "vehicle_count": 3,  "distance_cm": 80 }
      }
    },
    {
      "id": "evening_rush",
      "name": "Evening Commute Rush",
      "description": "Heavy outbound traffic on Eastbound (Lane 2) and Westbound (Lane 4).",
      "lanes": {
        "lane1": { "vehicle_count": 5,  "distance_cm": 70 },
        "lane2": { "vehicle_count": 25, "distance_cm": 8 },
        "lane3": { "vehicle_count": 6,  "distance_cm": 65 },
        "lane4": { "vehicle_count": 20, "distance_cm": 12 }
      }
    },
    {
      "id": "emergency_ambulance",
      "name": "Ambulance Priority Dispatch",
      "description": "Emergency vehicle approaching on Lane 2 (Eastbound). Controller triggers preemption.",
      "emergency_lane": 2,
      "lanes": {
        "lane1": { "vehicle_count": 10, "distance_cm": 45 },
        "lane2": { "vehicle_count": 15, "distance_cm": 25 },
        "lane3": { "vehicle_count": 8,  "distance_cm": 55 },
        "lane4": { "vehicle_count": 9,  "distance_cm": 50 }
      }
    },
    {
      "id": "midnight_lull",
      "name": "Midnight / Low Density",
      "description": "Minimal sparse traffic across all lanes, system uses minimum green cycle times.",
      "lanes": {
        "lane1": { "vehicle_count": 1, "distance_cm": 95 },
        "lane2": { "vehicle_count": 2, "distance_cm": 90 },
        "lane3": { "vehicle_count": 0, "distance_cm": 100 },
        "lane4": { "vehicle_count": 1, "distance_cm": 92 }
      }
    }
  ]
}
```

Code Listing: 5. Node-RED Dynamic Logic Flow

Path: node_red/flows.json | **Type:** Node-RED Flow JSON

```
[
  {
    "id": "tab_smart_traffic",
    "type": "tab",
    "label": "Smart Traffic Controller",
    "disabled": false,
    "info": "Dynamic Adaptive Smart Traffic Light System with HiveMQ MQTT integration, Synthetic & Real ESP32 data handling, and
  },
  {
    "id": "mqtt_hivemq_broker",
    "type": "mqtt-broker",
    "name": "HiveMQ Public Broker",
    "broker": "broker.hivemq.com",
    "port": "1883",
    "clientid": "NodeRed_SmartTraffic_Core",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": "4",
    "keepalive": "60",
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "smart_traffic/demo/system/nodered_status",
    "birthQos": "0",
    "birthPayload": "{\"status\":\"ONLINE\",\"service\":\"Node-RED Dynamic Controller\"}",
    "birthMsg": {},
    "closeTopic": "smart_traffic/demo/system/nodered_status",
    "closeQos": "0",
    "closePayload": "{\"status\":\"OFFLINE\"}",
    "closeMsg": {},
    "willTopic": "smart_traffic/demo/system/nodered_status",
    "willQos": "0",
    "willPayload": "{\"status\":\"DISCONNECTED\"}",
    "willMsg": {},
    "userProps": "",
    "sessionExpiry": ""
  },
  {
    "id": "ui_tab_traffic",
    "type": "ui_tab",
    "name": "Traffic Control Center",
    "icon": "dashboard",
    "order": 1
  },
  {
    "id": "ui_group_status",
    "type": "ui_group",
    "name": "System Overview & Mode",
    "tab": "ui_tab_traffic",
    "order": 1,
    "disp": true,
    "width": "6",
    "collapse": false
  },
  {
    "id": "ui_group_lights",
    "type": "ui_group",
    "name": "Intersection Traffic Lights",
    "tab": "ui_tab_traffic",
    "order": 2,
    "disp": true,
    "width": "6",
    "collapse": false
```

```
  },
  {
    "id": "ui_group_densities",
    "type": "ui_group",
    "name": "Live Vehicle Densities (Queues)",
    "tab": "ui_tab_traffic",
    "order": 3,
    "disp": true,
    "width": "6",
    "collapse": false
  },
  {
    "id": "ui_group_emergency",
    "type": "ui_group",
    "name": "Emergency Preemption & Overrides",
    "tab": "ui_tab_traffic",
    "order": 4,
    "disp": true,
    "width": "6",
    "collapse": false
  },
  {
    "id": "mqtt_in_density",
    "type": "mqtt in",
    "z": "tab_smart_traffic",
    "name": "MQTT In: Lane Densities",
    "topic": "smart_traffic/demo/+/density",
    "qos": "0",
    "datatype": "auto-detect",
    "broker": "mqtt_hivemq_broker",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 170,
    "y": 140,
    "wires": [["fn_aggregate_densities"]]
  },
  {
    "id": "mqtt_in_mode",
    "type": "mqtt in",
    "z": "tab_smart_traffic",
    "name": "MQTT In: System Mode",
    "topic": "smart_traffic/demo/system/mode",
    "qos": "0",
    "datatype": "auto-detect",
    "broker": "mqtt_hivemq_broker",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 160,
    "y": 200,
    "wires": [["fn_system_mode"]]
  },
  {
    "id": "mqtt_in_emergency",
    "type": "mqtt in",
    "z": "tab_smart_traffic",
    "name": "MQTT In: Emergency",
    "topic": "smart_traffic/demo/control/emergency",
    "qos": "0",
```

```
    "datatype": "auto-detect",
    "broker": "mqtt_hivemq_broker",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 160,
    "y": 260,
    "wires": [["fn_emergency_handler"]]
  },
  {
    "id": "fn_aggregate_densities",
    "type": "function",
    "z": "tab_smart_traffic",
    "name": "Aggregate Lane Densities",
    "func": "var densities = flow.get('lane_densities') || { 1: 5, 2: 3, 3: 8, 4: 4 };\nvar distances = flow.get('lane_distances
    "outputs": 2,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 450,
    "y": 140,
    "wires": [["gauge_lane1", "gauge_lane2", "gauge_lane3", "gauge_lane4"], []]
  },
  {
    "id": "fn_system_mode",
    "type": "function",
    "z": "tab_smart_traffic",
    "name": "Update System Mode",
    "func": "var mode = 'SYNTHETIC';\nif (typeof msg.payload === 'object' && msg.payload.mode) {\n    mode = msg.payload.mode;\n
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 440,
    "y": 200,
    "wires": [["ui_text_mode"]]
  },
  {
    "id": "fn_emergency_handler",
    "type": "function",
    "z": "tab_smart_traffic",
    "name": "Process Emergency Override",
    "func": "var action = 'CLEAR';\nvar lane = 0;\nif (typeof msg.payload === 'object') {\n    action = msg.payload.action || '
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 460,
    "y": 260,
    "wires": [["ui_text_emergency"]]
  },
  {
    "id": "ticker_1sec",
    "type": "inject",
    "z": "tab_smart_traffic",
```

```
    "name": "1-Sec Clock Ticker",
    "props": [{ "p": "payload" }],
    "repeat": "1",
    "crontab": "",
    "once": true,
    "onceDelay": 0.5,
    "topic": "",
    "payload": "tick",
    "payloadType": "str",
    "x": 160,
    "y": 380,
    "wires": [["fn_traffic_controller_engine"]]
  },
  {
    "id": "fn_traffic_controller_engine",
    "type": "function",
    "z": "tab_smart_traffic",
    "name": "Dynamic Adaptive Algorithm Engine",
    "func": "// Adaptive Traffic Control Logic\nvar state = flow.get('traffic_state') || {\n    active_lane: 1,\n    phase: 'GRE
    "outputs": 3,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 480,
    "y": 380,
    "wires": [
      ["mqtt_out_lights_json", "ui_light_states", "ui_progress_timer"],
      ["mqtt_out_lane_lights"],
      ["ui_text_active_lane"]
    ]
  },
  {
    "id": "mqtt_out_lights_json",
    "type": "mqtt out",
    "z": "tab_smart_traffic",
    "name": "MQTT Out: Master Lights JSON",
    "topic": "smart_traffic/demo/control/lights",
    "qos": "0",
    "retain": "true",
    "respTopic": "",
    "contentType": "",
    "userProps": "",
    "correl": "",
    "expiry": "",
    "broker": "mqtt_hivemq_broker",
    "x": 840,
    "y": 340,
    "wires": []
  },
  {
    "id": "mqtt_out_lane_lights",
    "type": "mqtt out",
    "z": "tab_smart_traffic",
    "name": "MQTT Out: Discrete Lane Lights",
    "topic": "",
    "qos": "0",
    "retain": "false",
    "respTopic": "",
    "contentType": "",
    "userProps": "",
```

```
    "correl": "",
    "expiry": "",
    "broker": "mqtt_hivemq_broker",
    "x": 840,
    "y": 400,
    "wires": []
  },
  {
    "id": "ui_text_mode",
    "type": "ui_text",
    "z": "tab_smart_traffic",
    "group": "ui_group_status",
    "order": 1,
    "width": "6",
    "height": "1",
    "name": "System Mode Text",
    "label": "Data Source Mode:",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "className": "",
    "x": 750,
    "y": 200,
    "wires": []
  },
  {
    "id": "ui_text_emergency",
    "type": "ui_text",
    "z": "tab_smart_traffic",
    "group": "ui_group_status",
    "order": 2,
    "width": "6",
    "height": "1",
    "name": "Emergency State Text",
    "label": "Emergency Status:",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "className": "",
    "x": 760,
    "y": 260,
    "wires": []
  },
  {
    "id": "ui_text_active_lane",
    "type": "ui_text",
    "z": "tab_smart_traffic",
    "group": "ui_group_status",
    "order": 3,
    "width": "6",
    "height": "1",
    "name": "Active Lane & Countdown",
    "label": "Active Lane (Countdown):",
    "format": "Lane {{msg.payload.active_lane}} ({{msg.payload.remaining}}s left)",
    "layout": "row-spread",
    "className": "",
    "x": 790,
    "y": 460,
    "wires": []
  },
  {
    "id": "ui_progress_timer",
    "type": "ui_gauge",
    "z": "tab_smart_traffic",
```

```
    "name": "Seconds Remaining",
    "group": "ui_group_status",
    "order": 4,
    "width": "6",
    "height": "3",
    "gtype": "donut",
    "title": "Phase Countdown (Seconds)",
    "label": "sec",
    "format": "{{msg.payload.remaining_sec}}",
    "min": 0,
    "max": 35,
    "colors": ["#00B500", "#E6E600", "#CA3838"],
    "seg1": "",
    "seg2": "",
    "className": "",
    "x": 800,
    "y": 500,
    "wires": []
  },
  {
    "id": "ui_light_states",
    "type": "ui_template",
    "z": "tab_smart_traffic",
    "group": "ui_group_lights",
    "name": "4-Way Light Status Display",
    "order": 1,
    "width": "6",
    "height": "4",
    "format": "<div style='display: grid; grid-template-columns: 1fr 1fr; gap: 10px; font-family: sans-serif; text-align: center
    "storeOutMessages": true,
    "fwdInMessages": true,
    "resendOnRefresh": true,
    "templateScope": "local",
    "className": "",
    "x": 800,
    "y": 550,
    "wires": [[]]
  },
  {
    "id": "gauge_lane1",
    "type": "ui_gauge",
    "z": "tab_smart_traffic",
    "name": "Lane 1 Density Gauge",
    "group": "ui_group_densities",
    "order": 1,
    "width": "3",
    "height": "2",
    "gtype": "gage",
    "title": "Lane 1 (North)",
    "label": "cars",
    "format": "{{msg.payload}}",
    "min": 0,
    "max": 30,
    "colors": ["#00B500", "#E6E600", "#CA3838"],
    "seg1": "8",
    "seg2": "18",
    "className": "",
    "x": 750,
    "y": 60,
    "wires": []
  },
  {
```

```
    "id": "gauge_lane2",
    "type": "ui_gauge",
    "z": "tab_smart_traffic",
    "name": "Lane 2 Density Gauge",
    "group": "ui_group_densities",
    "order": 2,
    "width": "3",
    "height": "2",
    "gtype": "gage",
    "title": "Lane 2 (East)",
    "label": "cars",
    "format": "{{msg.payload}}",
    "min": 0,
    "max": 30,
    "colors": ["#00B500", "#E6E600", "#CA3838"],
    "seg1": "8",
    "seg2": "18",
    "className": "",
    "x": 750,
    "y": 100,
    "wires": []
  },
  {
    "id": "gauge_lane3",
    "type": "ui_gauge",
    "z": "tab_smart_traffic",
    "name": "Lane 3 Density Gauge",
    "group": "ui_group_densities",
    "order": 3,
    "width": "3",
    "height": "2",
    "gtype": "gage",
    "title": "Lane 3 (South)",
    "label": "cars",
    "format": "{{msg.payload}}",
    "min": 0,
    "max": 30,
    "colors": ["#00B500", "#E6E600", "#CA3838"],
    "seg1": "8",
    "seg2": "18",
    "className": "",
    "x": 750,
    "y": 140,
    "wires": []
  },
  {
    "id": "gauge_lane4",
    "type": "ui_gauge",
    "z": "tab_smart_traffic",
    "name": "Lane 4 Density Gauge",
    "group": "ui_group_densities",
    "order": 4,
    "width": "3",
    "height": "2",
    "gtype": "gage",
    "title": "Lane 4 (West)",
    "label": "cars",
    "format": "{{msg.payload}}",
    "min": 0,
    "max": 30,
    "colors": ["#00B500", "#E6E600", "#CA3838"],
    "seg1": "8",
```

```
    "seg2": "18",
    "className": "",
    "x": 750,
    "y": 180,
    "wires": []
  },
  {
    "id": "ui_btn_emg_l1",
    "type": "ui_button",
    "z": "tab_smart_traffic",
    "name": "Emergency Lane 1",
    "group": "ui_group_emergency",
    "order": 1,
    "width": "3",
    "height": "1",
    "passthru": false,
    "label": "■ Priority Lane 1",
    "tooltip": "Clear path for ambulance on Lane 1",
    "color": "#ffffff",
    "bgcolor": "#dc2626",
    "className": "",
    "icon": "",
    "payload": "{\"action\":\"OVERRIDE\",\"lane\":1,\"source\":\"NodeRED_Dashboard\"}",
    "payloadType": "json",
    "topic": "smart_traffic/demo/control/emergency",
    "topicType": "str",
    "x": 160,
    "y": 480,
    "wires": [["mqtt_out_emergency_cmd"]]
  },
  {
    "id": "ui_btn_emg_l2",
    "type": "ui_button",
    "z": "tab_smart_traffic",
    "name": "Emergency Lane 2",
    "group": "ui_group_emergency",
    "order": 2,
    "width": "3",
    "height": "1",
    "passthru": false,
    "label": "■ Priority Lane 2",
    "tooltip": "Clear path for ambulance on Lane 2",
    "color": "#ffffff",
    "bgcolor": "#dc2626",
    "className": "",
    "icon": "",
    "payload": "{\"action\":\"OVERRIDE\",\"lane\":2,\"source\":\"NodeRED_Dashboard\"}",
    "payloadType": "json",
    "topic": "smart_traffic/demo/control/emergency",
    "topicType": "str",
    "x": 160,
    "y": 520,
    "wires": [["mqtt_out_emergency_cmd"]]
  },
  {
    "id": "ui_btn_emg_clear",
    "type": "ui_button",
    "z": "tab_smart_traffic",
    "name": "Clear Emergency",
    "group": "ui_group_emergency",
    "order": 3,
    "width": "6",
```

```
    "height": "1",
    "passthru": false,
    "label": "✓ Clear Emergency (Resume Auto)",
    "tooltip": "Resume standard adaptive cycle",
    "color": "#ffffff",
    "bgcolor": "#16a34a",
    "className": "",
    "icon": "",
    "payload": "{\"action\":\"CLEAR\",\"lane\":0,\"source\":\"NodeRED_Dashboard\"}",
    "payloadType": "json",
    "topic": "smart_traffic/demo/control/emergency",
    "topicType": "str",
    "x": 170,
    "y": 560,
    "wires": [["mqtt_out_emergency_cmd"]]
  },
  {
    "id": "mqtt_out_emergency_cmd",
    "type": "mqtt out",
    "z": "tab_smart_traffic",
    "name": "MQTT Out: Emergency Command",
    "topic": "smart_traffic/demo/control/emergency",
    "qos": "0",
    "retain": "false",
    "respTopic": "",
    "contentType": "",
    "userProps": "",
    "correl": "",
    "expiry": "",
    "broker": "mqtt_hivemq_broker",
    "x": 510,
    "y": 520,
    "wires": []
  }
]
```

Code Listing: 6. Live Web Dashboard UI

Path: index.html | **Type:** HTML5 & Tailwind CSS

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Smart Traffic Light Control System - Cloud Dashboard</title>
  <!-- Tailwind CSS -->
  <script src="https://cdn.tailwindcss.com"></script>
  <!-- FontAwesome Icons -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <!-- MQTT.js library for WebSockets -->
  <script src="https://unpkg.com/mqtt@5.1.4/dist/mqtt.min.js"></script>
  <link rel="stylesheet" href="css/style.css">
</head>
<body class="bg-slate-950 text-slate-100 min-h-screen font-sans antialiased selection:bg-emerald-500 selection:text-white">

  <!-- TOP NAVIGATION BAR -->
  <header class="border-b border-slate-800 bg-slate-900/80 backdrop-blur sticky top-0 z-50 px-4 lg:px-8 py-3.5 flex flex-wrap it
    <div class="flex items-center space-x-3">
      <div class="w-10 h-10 rounded-xl bg-gradient-to-tr from-emerald-500 to-teal-400 flex items-center justify-center shadow-lg
        <i class="fa-solid fa-traffic-light"></i>
      </div>
      <div>
        <h1 class="text-lg font-bold tracking-tight text-white flex items-center gap-2">
          Smart Traffic Light Control System
          <span class="text-xs font-semibold px-2 py-0.5 rounded-full bg-emerald-500/10 text-emerald-400 border border-emerald-5
        </h1>
        <p class="text-xs text-slate-400">HiveMQ MQTT • Dual Mode (Synthetic & ESP32 Sensors) • Adaptive AI Timing</p>
      </div>
    </div>

    <!-- CONNECTION STATUS & CONTROLS -->
    <div class="flex items-center space-x-3 text-sm">
      <div id="connectionStatusBadge" class="flex items-center space-x-2 px-3 py-1.5 rounded-lg bg-slate-800/80 border border-sl
        <span class="relative flex h-2.5 w-2.5">
          <span id="statusPing" class="animate-ping absolute inline-flex h-full w-full rounded-full bg-amber-400 opacity-75"></s
          <span id="statusDot" class="relative inline-flex rounded-full h-2.5 w-2.5 bg-amber-500"></span>
        </span>
        <span id="statusText" class="text-xs font-mono font-medium">Connecting to HiveMQ...</span>
      </div>

      <button id="btnOpenSettings" class="px-3 py-1.5 text-xs font-medium rounded-lg bg-slate-800 hover:bg-slate-700 border bord
        <i class="fa-solid fa-gear"></i> Settings
      </button>

      <a href="https://github.com" target="_blank" id="repoLink" class="px-3 py-1.5 text-xs font-medium rounded-lg bg-emerald-60
        <i class="fa-brands fa-github"></i> GitHub Repo
      </a>
    </div>
  </header>

  <!-- EMERGENCY ALERT BANNER (HIDDEN BY DEFAULT) -->
  <div id="emergencyBanner" class="hidden bg-gradient-to-r from-red-600 via-rose-600 to-red-600 text-white px-4 py-2.5 text-cent
    <i class="fa-solid fa-triangle-exclamation text-lg"></i>
    <span>PRIORITY OVERRIDE DETECTED: <span id="emergencyBannerText">EMERGENCY VEHICLE INBOUND ON LANE 1</span></span>
    <button id="btnBannerClear" class="ml-4 px-2.5 py-1 text-xs bg-black/40 hover:bg-black/60 rounded border border-white/20 tra
      Dismiss / Resume
    </button>
  </div>

  <!-- MAIN CONTAINER -->
  <main class="max-w-7xl mx-auto px-4 lg:px-8 py-6 space-y-6">
```

```
<!-- TOP STATS ROW -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
  <!-- Active Mode Card -->
  <div class="bg-slate-900/60 border border-slate-800 rounded-xl p-4 flex items-center justify-between">
    <div>
      <span class="text-xs font-medium text-slate-400 uppercase tracking-wider">Telemetry Source</span>
      <div id="statMode" class="text-lg font-bold text-teal-400 mt-1 flex items-center gap-2">
        <i class="fa-solid fa-microchip"></i> <span>SYNTHETIC</span>
      </div>
      <span id="statSourceDetail" class="text-xs text-slate-500">Python Simulator</span>
    </div>
    <div class="flex flex-col gap-1">
      <button id="btnSwitchModeSynthetic" class="px-2 py-1 text-xs font-semibold rounded bg-teal-500/20 border border-teal-5
        Synthetic
      </button>
      <button id="btnSwitchModeReal" class="px-2 py-1 text-xs font-semibold rounded bg-slate-800 border border-slate-700 tex
        ESP32 Sensor
      </button>
    </div>
  </div>

  <!-- Current Green Phase Card -->
  <div class="bg-slate-900/60 border border-slate-800 rounded-xl p-4 flex items-center justify-between">
    <div>
      <span class="text-xs font-medium text-slate-400 uppercase tracking-wider">Active Green Phase</span>
      <div id="statActiveLane" class="text-lg font-bold text-emerald-400 mt-1">
        Lane 1 (North)
      </div>
      <span id="statPhaseState" class="text-xs text-emerald-400/80 font-mono">Status: GREEN</span>
    </div>
    <div class="w-12 h-12 rounded-xl bg-emerald-500/10 border border-emerald-500/30 flex items-center justify-center text-em
      1
    </div>
  </div>

  <!-- Phase Countdown Card -->
  <div class="bg-slate-900/60 border border-slate-800 rounded-xl p-4 flex items-center justify-between">
    <div>
      <span class="text-xs font-medium text-slate-400 uppercase tracking-wider">Phase Countdown</span>
      <div class="text-2xl font-black font-mono text-white mt-1 flex items-baseline gap-1">
        <span id="statCountdown">15</span>
        <span class="text-xs font-normal text-slate-400">sec</span>
      </div>
      <div class="w-32 bg-slate-800 h-1.5 rounded-full overflow-hidden mt-1.5">
        <div id="statProgressBar" class="bg-emerald-500 h-full w-1/2 transition-all duration-300"></div>
      </div>
    </div>
    <div class="w-12 h-12 rounded-xl bg-slate-800 border border-slate-700 flex items-center justify-center text-slate-300 te
      <i class="fa-solid fa-stopwatch"></i>
    </div>
  </div>

  <!-- Total Intersection Vehicles -->
  <div class="bg-slate-900/60 border border-slate-800 rounded-xl p-4 flex items-center justify-between">
    <div>
      <span class="text-xs font-medium text-slate-400 uppercase tracking-wider">Total Intersection Queue</span>
      <div class="text-2xl font-black font-mono text-white mt-1 flex items-baseline gap-1">
        <span id="statTotalVehicles">20</span>
        <span class="text-xs font-normal text-slate-400">vehicles</span>
      </div>
      <span id="statWaitLevel" class="text-xs text-amber-400 font-medium">Moderate Flow</span>
```

```
        </div>
        <div class="w-12 h-12 rounded-xl bg-slate-800 border border-slate-700 flex items-center justify-center text-slate-300 te
          <i class="fa-solid fa-car-side"></i>
        </div>
      </div>
    </div>

    <!-- MAIN TWO COLUMN GRID -->
    <div class="grid grid-cols-1 lg:grid-cols-12 gap-6">

      <!-- LEFT COLUMN: 4-WAY INTERSECTION CANVAS SIMULATION -->
      <section class="lg:col-span-7 bg-slate-900/80 border border-slate-800 rounded-2xl p-5 shadow-xl flex flex-col justify-betw
        <div class="flex items-center justify-between border-b border-slate-800/80 pb-3 mb-4">
          <div>
            <h2 class="text-base font-bold text-white flex items-center gap-2">
              <i class="fa-solid fa-map-location-dot text-emerald-400"></i>
              Real-Time 4-Way Intersection Monitor
            </h2>
            <p class="text-xs text-slate-400">Live animated traffic rendering synchronized with MQTT state</p>
          </div>
          <span class="text-xs font-mono text-slate-400 bg-slate-800/80 px-2.5 py-1 rounded-md border border-slate-700">
            FPS: <span id="fpsCount">60</span>
          </span>
        </div>

        <!-- Intersection Canvas Container -->
        <div class="relative w-full aspect-square max-h-[500px] bg-slate-950 rounded-xl border border-slate-800/80 overflow-hid
          <canvas id="intersectionCanvas" width="600" height="600" class="w-full h-full object-contain"></canvas>

          <!-- Quick Canvas Legend -->
          <div class="absolute bottom-2 left-2 bg-slate-900/90 backdrop-blur px-2.5 py-1.5 rounded-lg border border-slate-800 te
            <div class="flex items-center gap-1.5"><span class="w-2 h-2 rounded-full bg-emerald-500"></span> Green (Moving)</div
            <div class="flex items-center gap-1.5"><span class="w-2 h-2 rounded-full bg-amber-400"></span> Yellow (Slow/Stop)</d
            <div class="flex items-center gap-1.5"><span class="w-2 h-2 rounded-full bg-red-500"></span> Red (Queueing)</div>
          </div>
        </div>

        <!-- Quick Manual Light Overrides -->
        <div class="mt-4 pt-3 border-t border-slate-800/80 flex flex-wrap items-center justify-between gap-3 text-xs">
          <span class="text-slate-400 font-medium">Quick Direct Control:</span>
          <div class="flex items-center gap-2">
            <button onclick="window.TrafficApp.forceLaneGreen(1)" class="px-2.5 py-1 bg-slate-800 hover:bg-slate-700 rounded tex
            <button onclick="window.TrafficApp.forceLaneGreen(2)" class="px-2.5 py-1 bg-slate-800 hover:bg-slate-700 rounded tex
            <button onclick="window.TrafficApp.forceLaneGreen(3)" class="px-2.5 py-1 bg-slate-800 hover:bg-slate-700 rounded tex
            <button onclick="window.TrafficApp.forceLaneGreen(4)" class="px-2.5 py-1 bg-slate-800 hover:bg-slate-700 rounded tex
            <button onclick="window.TrafficApp.forceAllRed()" class="px-2.5 py-1 bg-red-950/60 hover:bg-red-900/80 text-red-300
            <button onclick="window.TrafficApp.resumeAuto()" class="px-2.5 py-1 bg-emerald-950/60 hover:bg-emerald-900/80 text-e
          </div>
        </div>
      </section>

      <!-- RIGHT COLUMN: 4 LANES TELEMETRY + CONTROLS -->
      <section class="lg:col-span-5 space-y-6">

        <!-- 4 LANES TELEMETRY CARDS -->
        <div class="bg-slate-900/80 border border-slate-800 rounded-2xl p-5 shadow-xl">
          <div class="flex items-center justify-between border-b border-slate-800/80 pb-3 mb-4">
            <h2 class="text-base font-bold text-white flex items-center gap-2">
              <i class="fa-solid fa-road text-teal-400"></i>
              Lane Densities & Ultrasonic Readings
            </h2>
            <span class="text-xs text-slate-400">4 Approaches</span>
```

```
        </div>

        <div class="space-y-3">
          <!-- Lane 1 -->
          <div id="laneCard1" class="p-3 bg-slate-950/60 rounded-xl border border-slate-800/80 transition-all">
            <div class="flex items-center justify-between mb-2">
              <div class="flex items-center gap-2">
                <span class="w-6 h-6 rounded bg-slate-800 text-white text-xs font-bold flex items-center justify-center">1</sp
                <span class="font-bold text-sm text-slate-200">Lane 1 (North)</span>
                <span id="lane1Badge" class="text-[10px] font-mono px-2 py-0.5 rounded-full bg-emerald-500/10 text-emerald-40
              </div>
              <div class="text-xs text-slate-400 font-mono">
                <span id="lane1Dist">35</span> cm | <span id="lane1Cars" class="font-bold text-white">5</span> cars
              </div>
            </div>
            <div class="w-full bg-slate-800 h-2 rounded-full overflow-hidden">
              <div id="lane1Bar" class="bg-emerald-500 h-full w-[25%] transition-all duration-300"></div>
            </div>
          </div>

          <!-- Lane 2 -->
          <div id="laneCard2" class="p-3 bg-slate-950/60 rounded-xl border border-slate-800/80 transition-all">
            <div class="flex items-center justify-between mb-2">
              <div class="flex items-center gap-2">
                <span class="w-6 h-6 rounded bg-slate-800 text-white text-xs font-bold flex items-center justify-center">2</sp
                <span class="font-bold text-sm text-slate-200">Lane 2 (East)</span>
                <span id="lane2Badge" class="text-[10px] font-mono px-2 py-0.5 rounded-full bg-red-500/10 text-red-400 border
              </div>
              <div class="text-xs text-slate-400 font-mono">
                <span id="lane2Dist">50</span> cm | <span id="lane2Cars" class="font-bold text-white">3</span> cars
              </div>
            </div>
            <div class="w-full bg-slate-800 h-2 rounded-full overflow-hidden">
              <div id="lane2Bar" class="bg-red-500 h-full w-[15%] transition-all duration-300"></div>
            </div>
          </div>

          <!-- Lane 3 -->
          <div id="laneCard3" class="p-3 bg-slate-950/60 rounded-xl border border-slate-800/80 transition-all">
            <div class="flex items-center justify-between mb-2">
              <div class="flex items-center gap-2">
                <span class="w-6 h-6 rounded bg-slate-800 text-white text-xs font-bold flex items-center justify-center">3</sp
                <span class="font-bold text-sm text-slate-200">Lane 3 (South)</span>
                <span id="lane3Badge" class="text-[10px] font-mono px-2 py-0.5 rounded-full bg-red-500/10 text-red-400 border
              </div>
              <div class="text-xs text-slate-400 font-mono">
                <span id="lane3Dist">20</span> cm | <span id="lane3Cars" class="font-bold text-white">8</span> cars
              </div>
            </div>
            <div class="w-full bg-slate-800 h-2 rounded-full overflow-hidden">
              <div id="lane3Bar" class="bg-red-500 h-full w-[40%] transition-all duration-300"></div>
            </div>
          </div>

          <!-- Lane 4 -->
          <div id="laneCard4" class="p-3 bg-slate-950/60 rounded-xl border border-slate-800/80 transition-all">
            <div class="flex items-center justify-between mb-2">
              <div class="flex items-center gap-2">
                <span class="w-6 h-6 rounded bg-slate-800 text-white text-xs font-bold flex items-center justify-center">4</sp
                <span class="font-bold text-sm text-slate-200">Lane 4 (West)</span>
                <span id="lane4Badge" class="text-[10px] font-mono px-2 py-0.5 rounded-full bg-red-500/10 text-red-400 border
              </div>
```

```
            <div class="text-xs text-slate-400 font-mono">
              <span id="lane4Dist">42</span> cm | <span id="lane4Cars" class="font-bold text-white">4</span> cars
            </div>
          </div>
          <div class="w-full bg-slate-800 h-2 rounded-full overflow-hidden">
            <div id="lane4Bar" class="bg-red-500 h-full w-[20%] transition-all duration-300"></div>
          </div>
        </div>
      </div>
    </div>

    <!-- EMERGENCY VEHICLE PREEMPTION DISPATCH -->
    <div class="bg-slate-900/80 border border-slate-800 rounded-2xl p-5 shadow-xl">
      <div class="flex items-center justify-between border-b border-slate-800/80 pb-3 mb-4">
        <h2 class="text-base font-bold text-white flex items-center gap-2">
          <i class="fa-solid fa-truck-medical text-rose-500"></i>
          Emergency Vehicle Preemption
        </h2>
        <span class="text-xs text-rose-400 font-medium">Priority Override</span>
      </div>

      <p class="text-xs text-slate-400 mb-3">
        Click any lane to simulate an approaching Ambulance or Fire Truck. The AI controller immediately grants green light
      </p>

      <div class="grid grid-cols-2 gap-2.5">
        <button onclick="window.TrafficApp.triggerEmergency(1)" class="px-3 py-2.5 rounded-xl bg-rose-600/10 hover:bg-rose-6
          <i class="fa-solid fa-truck-medical"></i> Priority Lane 1
        </button>
        <button onclick="window.TrafficApp.triggerEmergency(2)" class="px-3 py-2.5 rounded-xl bg-rose-600/10 hover:bg-rose-6
          <i class="fa-solid fa-truck-medical"></i> Priority Lane 2
        </button>
        <button onclick="window.TrafficApp.triggerEmergency(3)" class="px-3 py-2.5 rounded-xl bg-rose-600/10 hover:bg-rose-6
          <i class="fa-solid fa-truck-medical"></i> Priority Lane 3
        </button>
        <button onclick="window.TrafficApp.triggerEmergency(4)" class="px-3 py-2.5 rounded-xl bg-rose-600/10 hover:bg-rose-6
          <i class="fa-solid fa-truck-medical"></i> Priority Lane 4
        </button>
      </div>

      <button onclick="window.TrafficApp.clearEmergency()" class="w-full mt-2.5 px-3 py-2 rounded-xl bg-emerald-600/20 hover
        <i class="fa-solid fa-check"></i> Clear Priority & Resume Adaptive Cycle
      </button>
    </div>

    <!-- SIMULATION SCENARIO PRESETS -->
    <div class="bg-slate-900/80 border border-slate-800 rounded-2xl p-5 shadow-xl">
      <div class="flex items-center justify-between border-b border-slate-800/80 pb-3 mb-4">
        <h2 class="text-base font-bold text-white flex items-center gap-2">
          <i class="fa-solid fa-sliders text-amber-400"></i>
          Synthetic Traffic Presets
        </h2>
        <span class="text-xs text-slate-400">Test Scenarios</span>
      </div>

      <div class="grid grid-cols-3 gap-2 text-xs">
        <button onclick="window.TrafficApp.applyPreset('morning')" class="p-2.5 rounded-xl bg-slate-800/80 hover:bg-slate-70
          <i class="fa-solid fa-sun text-amber-400 mb-1 block text-sm"></i>
          Morning Rush
        </button>
        <button onclick="window.TrafficApp.applyPreset('evening')" class="p-2.5 rounded-xl bg-slate-800/80 hover:bg-slate-70
          <i class="fa-solid fa-moon text-indigo-400 mb-1 block text-sm"></i>
```

```
            Evening Rush
          </button>
          <button onclick="window.TrafficApp.applyPreset('lull')" class="p-2.5 rounded-xl bg-slate-800/80 hover:bg-slate-700 b
            <i class="fa-solid fa-leaf text-teal-400 mb-1 block text-sm"></i>
            Midnight Lull
          </button>
        </div>
      </div>

    </section>
  </div>

  <!-- MQTT CONSOLE & LOGS -->
  <section class="bg-slate-900/80 border border-slate-800 rounded-2xl p-5 shadow-xl">
    <div class="flex items-center justify-between border-b border-slate-800/80 pb-3 mb-3">
      <div class="flex items-center gap-2">
        <i class="fa-solid fa-terminal text-emerald-400"></i>
        <h3 class="text-sm font-bold text-white">Live MQTT Telemetry Stream (HiveMQ)</h3>
        <span class="text-[11px] text-slate-500 font-mono" id="topicBaseDisplay">Topic: smart_traffic/demo/#</span>
      </div>
      <div class="flex items-center gap-2">
        <button id="btnClearLogs" class="text-xs text-slate-400 hover:text-slate-200 px-2 py-1 rounded bg-slate-800 border bor
          Clear Logs
        </button>
      </div>
    </div>
    <div id="mqttLogContainer" class="h-32 bg-slate-950 rounded-xl p-3 font-mono text-xs text-slate-300 overflow-y-auto space-
      <div class="text-slate-500 italic">[Waiting for MQTT packets...]</div>
    </div>
  </section>

</main>

<!-- MODAL: SETTINGS -->
<div id="settingsModal" class="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 hidden flex items-center justify-center p-4">
  <div class="bg-slate-900 border border-slate-800 rounded-2xl max-w-md w-full p-6 space-y-4 shadow-2xl">
    <div class="flex items-center justify-between border-b border-slate-800 pb-3">
      <h3 class="font-bold text-white text-base flex items-center gap-2">
        <i class="fa-solid fa-gear text-emerald-400"></i> MQTT Broker & Connection Settings
      </h3>
      <button id="btnCloseSettings" class="text-slate-400 hover:text-white transition">
        <i class="fa-solid fa-xmark text-lg"></i>
      </button>
    </div>

    <div class="space-y-3 text-sm">
      <div>
        <label class="block text-xs font-semibold text-slate-400 mb-1">WebSocket Broker URL / Host</label>
        <input type="text" id="cfgBrokerHost" value="broker.hivemq.com" class="w-full bg-slate-950 border border-slate-700 rou
        <p class="text-[10px] text-slate-500 mt-1">Default: broker.hivemq.com (Port 8884 for WSS)</p>
      </div>

      <div class="grid grid-cols-2 gap-3">
        <div>
          <label class="block text-xs font-semibold text-slate-400 mb-1">WSS Port</label>
          <input type="number" id="cfgBrokerPort" value="8884" class="w-full bg-slate-950 border border-slate-700 rounded-lg p
        </div>
        <div>
          <label class="block text-xs font-semibold text-slate-400 mb-1">Protocol Path</label>
          <input type="text" id="cfgBrokerPath" value="/mqtt" class="w-full bg-slate-950 border border-slate-700 rounded-lg px
        </div>
      </div>
```

```
        <div>
          <label class="block text-xs font-semibold text-slate-400 mb-1">Base Topic Prefix</label>
          <input type="text" id="cfgTopicPrefix" value="smart_traffic/demo" class="w-full bg-slate-950 border border-slate-700 r
          <p class="text-[10px] text-slate-500 mt-1">Must match in ESP32, Python Simulator, and Node-RED</p>
        </div>
      </div>

      <div class="pt-3 border-t border-slate-800 flex justify-end gap-2">
        <button id="btnSaveSettings" class="px-4 py-2 bg-emerald-600 hover:bg-emerald-500 text-white rounded-lg text-xs font-bo
          Save & Reconnect
        </button>
      </div>
    </div>
  </div>

  <!-- APPLICATION JAVASCRIPT -->
  <script src="js/app.js"></script>
</body>
</html>
```

Code Listing: 7. Web Dashboard JavaScript & Animation Engine

Path: js/app.js | **Type:** Vanilla JavaScript (MQTT.js & Canvas)

```
/**
 * Smart Traffic Light Control System - Web Dashboard Engine
 * Handles HiveMQ MQTT over WebSockets, HTML5 Canvas 4-Way Traffic Rendering,
 * Telemetry Visualization, and Remote Actuation.
 */

(function () {
  'use strict';

  // Configuration defaults
  const config = {
    brokerHost: localStorage.getItem('cfg_broker_host') || 'broker.hivemq.com',
    brokerPort: parseInt(localStorage.getItem('cfg_broker_port') || '8884', 10),
    brokerPath: localStorage.getItem('cfg_broker_path') || '/mqtt',
    topicPrefix: localStorage.getItem('cfg_topic_prefix') || 'smart_traffic/demo'
  };

  // State
  const state = {
    connected: false,
    systemMode: 'SYNTHETIC', // 'SYNTHETIC' or 'REAL_ESP32'
    activeLane: 1,
    phase: 'GREEN',
    remainingSec: 15,
    totalPhaseSec: 15,
    emergencyActive: false,
    emergencyLane: 0,
    lanes: {
      1: { name: 'North', light: 'GREEN', vehicles: 5, distance_cm: 35 },
      2: { name: 'East',  light: 'RED',   vehicles: 3, distance_cm: 50 },
      3: { name: 'South', light: 'RED',   vehicles: 8, distance_cm: 20 },
      4: { name: 'West',  light: 'RED',   vehicles: 4, distance_cm: 42 }
    },
    localSimulationActive: true // Runs client-side fallback cycle if external controller is offline
  };

  let mqttClient = null;
  let lastMqttControllerHeartbeat = 0;

  // DOM Elements
  const elStatusBadge = document.getElementById('connectionStatusBadge');
  const elStatusDot = document.getElementById('statusDot');
  const elStatusPing = document.getElementById('statusPing');
  const elStatusText = document.getElementById('statusText');

  const elStatMode = document.getElementById('statMode');
  const elStatSourceDetail = document.getElementById('statSourceDetail');
  const elStatActiveLane = document.getElementById('statActiveLane');
  const elStatPhaseState = document.getElementById('statPhaseState');
  const elStatCountdown = document.getElementById('statCountdown');
  const elStatProgressBar = document.getElementById('statProgressBar');
  const elStatTotalVehicles = document.getElementById('statTotalVehicles');
  const elStatWaitLevel = document.getElementById('statWaitLevel');

  const elEmergencyBanner = document.getElementById('emergencyBanner');
  const elEmergencyBannerText = document.getElementById('emergencyBannerText');

  const elMqttLogContainer = document.getElementById('mqttLogContainer');
  const elTopicBaseDisplay = document.getElementById('topicBaseDisplay');

  // Canvas context
  const canvas = document.getElementById('intersectionCanvas');
```

```
const ctx = canvas.getContext('2d');

// ============================================================================
// 1. MQTT CLIENT CONNECTION & HANDLERS
// ============================================================================
function initMqtt() {
  const clientId = 'WebDashboard_' + Math.random().toString(16).substr(2, 8);
  const wsUrl = `wss://${config.brokerHost}:${config.brokerPort}${config.brokerPath}`;

  logToConsole(`Connecting to HiveMQ WebSocket: ${wsUrl}...`, 'info');
  updateConnectionStatus('connecting');

  try {
    mqttClient = mqtt.connect(wsUrl, {
      clientId: clientId,
      clean: true,
      reconnectPeriod: 4000,
      connectTimeout: 10000,
    });

    mqttClient.on('connect', () => {
      logToConsole(`[✓] Connected to HiveMQ Broker (${config.brokerHost})`, 'success');
      updateConnectionStatus('connected');

      // Subscribe to all topics under the prefix
      const wildcardTopic = `${config.topicPrefix}/#`;
      mqttClient.subscribe(wildcardTopic, (err) => {
        if (!err) {
          logToConsole(`Subscribed to topic pattern: ${wildcardTopic}`, 'info');
        }
      });

      // Request system mode or announce presence
      publishMqtt(`${config.topicPrefix}/system/dashboard_status`, {
        client: clientId,
        status: 'ONLINE',
        timestamp: Date.now()
      });
    });

    mqttClient.on('message', onMqttMessage);

    mqttClient.on('error', (err) => {
      logToConsole(`[✗] MQTT Error: ${err.message}`, 'error');
      updateConnectionStatus('error');
    });

    mqttClient.on('close', () => {
      updateConnectionStatus('disconnected');
    });

  } catch (e) {
    logToConsole(`Failed to initialize MQTT: ${e.message}`, 'error');
    updateConnectionStatus('error');
  }
}

function onMqttMessage(topic, payload) {
  const rawMsg = payload.toString();
  let data;
  try {
    data = JSON.parse(rawMsg);
```

```
    } catch (e) {
      data = rawMsg;
    }

    logToConsole(`RX: ${topic} → ${typeof data === 'object' ? JSON.stringify(data) : data}`);

    // Route message by subtopic
    const subTopic = topic.replace(config.topicPrefix + '/', '');

    if (subTopic === 'control/lights') {
      lastMqttControllerHeartbeat = Date.now();
      state.localSimulationActive = false; // External controller (Node-RED/ESP32) is actively controlling

      if (typeof data === 'object') {
        state.activeLane = data.active_lane || state.activeLane;
        state.phase = data.phase || (data[`lane${state.activeLane}`] || 'GREEN');
        state.remainingSec = data.remaining_sec !== undefined ? data.remaining_sec : state.remainingSec;
        state.totalPhaseSec = data.total_phase_sec || 20;

        for (let i = 1; i <= 4; i++) {
          if (data[`lane${i}`]) {
            state.lanes[i].light = data[`lane${i}`];
          }
        }

        if (data.emergency_active !== undefined) {
          state.emergencyActive = data.emergency_active;
          state.emergencyLane = data.emergency_lane || 0;
        }

        updateUI();
      }
    } else if (subTopic.match(/^lane(\d)\/density$/)) {
      const match = subTopic.match(/^lane(\d)\/density$/);
      const laneId = parseInt(match[1], 10);
      if (typeof data === 'object') {
        state.lanes[laneId].vehicles = data.vehicle_count !== undefined ? data.vehicle_count : state.lanes[laneId].vehicles;
        state.lanes[laneId].distance_cm = data.distance_cm !== undefined ? data.distance_cm : state.lanes[laneId].distance_cm;
        if (data.source && data.source.includes('ESP32')) {
          setSystemMode('REAL_ESP32', false);
        }
      }
      updateUI();
    } else if (subTopic.match(/^lane(\d)\/light$/)) {
      const match = subTopic.match(/^lane(\d)\/light$/);
      const laneId = parseInt(match[1], 10);
      state.lanes[laneId].light = String(data).toUpperCase();
      updateUI();
    } else if (subTopic === 'control/emergency') {
      if (typeof data === 'object') {
        if (data.action === 'OVERRIDE') {
          state.emergencyActive = true;
          state.emergencyLane = data.lane || 1;
        } else if (data.action === 'CLEAR') {
          state.emergencyActive = false;
          state.emergencyLane = 0;
        }
      }
      updateUI();
    } else if (subTopic === 'system/mode') {
      const newMode = (typeof data === 'object' && data.mode) ? data.mode : String(data);
      setSystemMode(newMode, false);
```

```
  }
}

function publishMqtt(topic, payload, retain = false) {
  if (mqttClient && mqttClient.connected) {
    const strPayload = typeof payload === 'object' ? JSON.stringify(payload) : String(payload);
    mqttClient.publish(topic, strPayload, { qos: 0, retain: retain }, (err) => {
      if (!err) {
        logToConsole(`TX: ${topic} → ${strPayload}`);
      }
    });
  }
}

function updateConnectionStatus(status) {
  if (status === 'connected') {
    state.connected = true;
    elStatusDot.className = 'relative inline-flex rounded-full h-2.5 w-2.5 bg-emerald-500';
    elStatusPing.className = 'animate-ping absolute inline-flex h-full w-full rounded-full bg-emerald-400 opacity-75';
    elStatusText.textContent = `Connected (${config.brokerHost})`;
    elStatusText.className = 'text-xs font-mono font-medium text-emerald-400';
  } else if (status === 'connecting') {
    elStatusDot.className = 'relative inline-flex rounded-full h-2.5 w-2.5 bg-amber-500';
    elStatusPing.className = 'animate-ping absolute inline-flex h-full w-full rounded-full bg-amber-400 opacity-75';
    elStatusText.textContent = 'Connecting...';
    elStatusText.className = 'text-xs font-mono font-medium text-amber-400';
  } else {
    state.connected = false;
    elStatusDot.className = 'relative inline-flex rounded-full h-2.5 w-2.5 bg-red-500';
    elStatusPing.className = 'hidden';
    elStatusText.textContent = 'Disconnected';
    elStatusText.className = 'text-xs font-mono font-medium text-red-400';
  }
}

function logToConsole(message, type = 'normal') {
  const time = new Date().toTimeString().split(' ')[0];
  const line = document.createElement('div');
  if (type === 'success') line.className = 'text-emerald-400 font-semibold';
  else if (type === 'error') line.className = 'text-rose-400 font-semibold';
  else if (type === 'info') line.className = 'text-sky-400';
  else line.className = 'text-slate-400';

  line.textContent = `[${time}] ${message}`;
  elMqttLogContainer.appendChild(line);

  // Limit log lines to 150
  if (elMqttLogContainer.children.length > 150) {
    elMqttLogContainer.removeChild(elMqttLogContainer.firstChild);
  }
  elMqttLogContainer.scrollTop = elMqttLogContainer.scrollHeight;
}

// ============================================================================
// 2. FALLBACK INTERNAL CONTROLLER (STANDALONE DEMO MODE)
// ============================================================================
// Runs client-side cycle if Node-RED is not running yet
setInterval(() => {
  // If no remote controller message received within 5 seconds, fallback to local controller
  if (Date.now() - lastMqttControllerHeartbeat > 5000) {
    state.localSimulationActive = true;
  }
```

```
  if (state.localSimulationActive) {
    stepLocalTrafficCycle();
  }
}, 1000);

function stepLocalTrafficCycle() {
  state.remainingSec--;

  if (state.emergencyActive && state.emergencyLane >= 1 && state.emergencyLane <= 4) {
    // Emergency priority holds active lane
    if (state.activeLane !== state.emergencyLane) {
      state.activeLane = state.emergencyLane;
      state.phase = 'GREEN';
      state.remainingSec = 20;
    }
  } else if (state.remainingSec <= 0) {
    if (state.phase === 'GREEN') {
      state.phase = 'YELLOW';
      state.remainingSec = 3;
    } else {
      state.activeLane = (state.activeLane % 4) + 1;
      state.phase = 'GREEN';
      // Adaptive time based on queue
      const cars = state.lanes[state.activeLane].vehicles;
      state.totalPhaseSec = Math.min(30, Math.max(8, 8 + Math.round(cars * 1.5)));
      state.remainingSec = state.totalPhaseSec;
    }
  }

  // Update lane light states
  for (let i = 1; i <= 4; i++) {
    state.lanes[i].light = (i === state.activeLane) ? state.phase : 'RED';
  }

  updateUI();
}

// ============================================================================
// 3. UI STATE UPDATER
// ============================================================================
function updateUI() {
  // Stats cards
  const laneNames = { 1: 'Lane 1 (North)', 2: 'Lane 2 (East)', 3: 'Lane 3 (South)', 4: 'Lane 4 (West)' };
  elStatActiveLane.textContent = laneNames[state.activeLane];
  elStatPhaseState.textContent = `Status: ${state.phase}`;
  elStatCountdown.textContent = Math.max(0, state.remainingSec);

  // Progress bar
  const pct = Math.max(0, Math.min(100, (state.remainingSec / (state.totalPhaseSec || 15)) * 100));
  elStatProgressBar.style.width = `${pct}%`;
  elStatProgressBar.className = (state.phase === 'GREEN') ? 'bg-emerald-500 h-full transition-all duration-300' :
                                (state.phase === 'YELLOW') ? 'bg-amber-400 h-full transition-all duration-300' :
                                'bg-red-500 h-full transition-all duration-300';

  // Total vehicles
  let totalCars = 0;
  for (let i = 1; i <= 4; i++) totalCars += state.lanes[i].vehicles;
  elStatTotalVehicles.textContent = totalCars;
  if (totalCars < 10) {
    elStatWaitLevel.textContent = 'Light Flow';
    elStatWaitLevel.className = 'text-xs text-emerald-400 font-medium';
```

```
    } else if (totalCars < 24) {
      elStatWaitLevel.textContent = 'Moderate Flow';
      elStatWaitLevel.className = 'text-xs text-amber-400 font-medium';
    } else {
      elStatWaitLevel.textContent = 'Heavy Congestion';
      elStatWaitLevel.className = 'text-xs text-rose-400 font-medium';
    }

    // 4 Lane cards
    for (let i = 1; i <= 4; i++) {
      const lane = state.lanes[i];
      const card = document.getElementById(`laneCard${i}`);
      const badge = document.getElementById(`lane${i}Badge`);
      const dist = document.getElementById(`lane${i}Dist`);
      const cars = document.getElementById(`lane${i}Cars`);
      const bar = document.getElementById(`lane${i}Bar`);

      if (dist) dist.textContent = lane.distance_cm;
      if (cars) cars.textContent = lane.vehicles;

      const barPct = Math.min(100, Math.round((lane.vehicles / 25) * 100));
      if (bar) {
        bar.style.width = `${barPct}%`;
        bar.className = (lane.light === 'GREEN') ? 'bg-emerald-500 h-full transition-all duration-300' :
                        (lane.light === 'YELLOW') ? 'bg-amber-400 h-full transition-all duration-300' :
                        'bg-red-500 h-full transition-all duration-300';
      }

      if (badge) {
        badge.textContent = lane.light;
        if (lane.light === 'GREEN') {
          badge.className = 'text-[10px] font-mono px-2 py-0.5 rounded-full bg-emerald-500/10 text-emerald-400 border border-eme
          if (card) card.classList.add('active-lane-glow');
        } else if (lane.light === 'YELLOW') {
          badge.className = 'text-[10px] font-mono px-2 py-0.5 rounded-full bg-amber-500/10 text-amber-400 border border-amber-5
          if (card) card.classList.remove('active-lane-glow');
        } else {
          badge.textContent = 'RED';
          badge.className = 'text-[10px] font-mono px-2 py-0.5 rounded-full bg-red-500/10 text-red-400 border border-red-500/30
          if (card) card.classList.remove('active-lane-glow');
        }
      }
    }

    // Emergency Banner
    if (state.emergencyActive && state.emergencyLane > 0) {
      elEmergencyBanner.classList.remove('hidden');
      elEmergencyBannerText.textContent = `EMERGENCY VEHICLE APPROACHING ON LANE ${state.emergencyLane} (${laneNames[state.emerg
    } else {
      elEmergencyBanner.classList.add('hidden');
    }
  }

  function setSystemMode(mode, publish = true) {
    state.systemMode = mode;
    const btnSyn = document.getElementById('btnSwitchModeSynthetic');
    const btnReal = document.getElementById('btnSwitchModeReal');

    if (mode === 'REAL_ESP32') {
      elStatMode.innerHTML = '<i class="fa-solid fa-wifi text-emerald-400"></i> <span>ESP32 SENSORS</span>';
      elStatSourceDetail.textContent = 'Hardware HC-SR04 Nodes';
      btnReal.className = 'px-2 py-1 text-xs font-semibold rounded bg-emerald-500/20 border border-emerald-500/40 text-emerald-3
```

```
      btnSyn.className = 'px-2 py-1 text-xs font-semibold rounded bg-slate-800 border border-slate-700 text-slate-400 hover:text
    } else {
      elStatMode.innerHTML = '<i class="fa-solid fa-microchip text-teal-400"></i> <span>SYNTHETIC</span>';
      elStatSourceDetail.textContent = 'Python / Cloud Simulator';
      btnSyn.className = 'px-2 py-1 text-xs font-semibold rounded bg-teal-500/20 border border-teal-500/40 text-teal-300 hover:b
      btnReal.className = 'px-2 py-1 text-xs font-semibold rounded bg-slate-800 border border-slate-700 text-slate-400 hover:tex
    }

    if (publish) {
      publishMqtt(`${config.topicPrefix}/system/mode`, { mode: mode, timestamp: Date.now() }, true);
    }
  }

  // ============================================================================
  // 4. ANIMATED 2D 4-WAY INTERSECTION CANVAS
  // ============================================================================
  const cars = [];
  const CAR_COLORS = ['#38bdf8', '#fbbf24', '#f43f5e', '#a855f7', '#34d399', '#f97316'];

  class Vehicle {
    constructor(lane) {
      this.lane = lane; // 1: North (heading S), 2: East (heading W), 3: South (heading N), 4: West (heading E)
      this.color = CAR_COLORS[Math.floor(Math.random() * CAR_COLORS.length)];
      this.width = 18;
      this.length = 34;
      this.speed = 2.5 + Math.random() * 0.8;
      this.isEmergency = (state.emergencyActive && state.emergencyLane === lane && Math.random() < 0.4);

      if (this.isEmergency) {
        this.color = '#ef4444';
      }

      // Initialize position along road
      if (lane === 1) { // North -> South
        this.x = 280;
        this.y = -40;
        this.stopLine = 220;
      } else if (lane === 2) { // East -> West
        this.x = 640;
        this.y = 280;
        this.stopLine = 380;
      } else if (lane === 3) { // South -> North
        this.x = 320;
        this.y = 640;
        this.stopLine = 380;
      } else if (lane === 4) { // West -> East
        this.x = -40;
        this.y = 320;
        this.stopLine = 220;
      }
    }

    update(leadingCar) {
      const light = state.lanes[this.lane].light;
      const mustStop = (light === 'RED' || light === 'YELLOW');
      let targetSpeed = this.speed;

      // Distance to stop line or car ahead
      if (this.lane === 1) {
        if (mustStop && this.y < this.stopLine && this.y + 40 >= this.stopLine) {
          targetSpeed = 0;
        }
```

```
        if (leadingCar && (leadingCar.y - (this.y + this.length)) < 15) {
          targetSpeed = 0;
        }
        this.y += targetSpeed;
      } else if (this.lane === 2) {
        if (mustStop && this.x > this.stopLine && this.x - 40 <= this.stopLine) {
          targetSpeed = 0;
        }
        if (leadingCar && ((this.x - this.length) - leadingCar.x) < 15) {
          targetSpeed = 0;
        }
        this.x -= targetSpeed;
      } else if (this.lane === 3) {
        if (mustStop && this.y > this.stopLine && this.y - 40 <= this.stopLine) {
          targetSpeed = 0;
        }
        if (leadingCar && ((this.y - this.length) - leadingCar.y) < 15) {
          targetSpeed = 0;
        }
        this.y -= targetSpeed;
      } else if (this.lane === 4) {
        if (mustStop && this.x < this.stopLine && this.x + 40 >= this.stopLine) {
          targetSpeed = 0;
        }
        if (leadingCar && (leadingCar.x - (this.x + this.length)) < 15) {
          targetSpeed = 0;
        }
        this.x += targetSpeed;
      }
    }

    draw(ctx) {
      ctx.save();
      ctx.fillStyle = this.color;
      ctx.shadowColor = 'rgba(0, 0, 0, 0.4)';
      ctx.shadowBlur = 6;

      if (this.lane === 1 || this.lane === 3) {
        // Vertical cars
        ctx.beginPath();
        ctx.roundRect(this.x - this.width / 2, this.y - this.length / 2, this.width, this.length, 4);
        ctx.fill();

        // Windshields
        ctx.fillStyle = '#0f172a';
        ctx.fillRect(this.x - this.width / 2 + 2, this.y - 8, this.width - 4, 6);

        // Flashing light if emergency
        if (this.isEmergency) {
          ctx.fillStyle = (Math.floor(Date.now() / 150) % 2 === 0) ? '#38bdf8' : '#ef4444';
          ctx.beginPath();
          ctx.arc(this.x, this.y, 4, 0, Math.PI * 2);
          ctx.fill();
        }
      } else {
        // Horizontal cars
        ctx.beginPath();
        ctx.roundRect(this.x - this.length / 2, this.y - this.width / 2, this.length, this.width, 4);
        ctx.fill();

        // Windshields
        ctx.fillStyle = '#0f172a';
```

```
        ctx.fillRect(this.x - 8, this.y - this.width / 2 + 2, 6, this.width - 4);

        if (this.isEmergency) {
          ctx.fillStyle = (Math.floor(Date.now() / 150) % 2 === 0) ? '#38bdf8' : '#ef4444';
          ctx.beginPath();
          ctx.arc(this.x, this.y, 4, 0, Math.PI * 2);
          ctx.fill();
        }
      }

      ctx.restore();
    }

    isOutOfBounds() {
      return (this.x < -60 || this.x > 660 || this.y < -60 || this.y > 660);
    }
  }

  // Spawn cars based on lane vehicle density
  setInterval(() => {
    for (let lane = 1; lane <= 4; lane++) {
      const activeLaneCars = cars.filter(c => c.lane === lane);
      const targetCount = Math.min(10, Math.ceil(state.lanes[lane].vehicles / 2.5));

      if (activeLaneCars.length < targetCount && Math.random() < 0.6) {
        cars.push(new Vehicle(lane));
      }
    }
  }, 1200);

  function drawIntersection() {
    ctx.clearRect(0, 0, 600, 600);

    // 1. Grass / Ground Background
    ctx.fillStyle = '#0f172a';
    ctx.fillRect(0, 0, 600, 600);

    // Corner lawns
    ctx.fillStyle = '#1e293b';
    ctx.fillRect(0, 0, 240, 240);
    ctx.fillRect(360, 0, 240, 240);
    ctx.fillRect(0, 360, 240, 240);
    ctx.fillRect(360, 360, 240, 240);

    // 2. Asphalt Roads
    ctx.fillStyle = '#1e293b';
    // Vertical Road (Lanes 1 & 3)
    ctx.fillRect(240, 0, 120, 600);
    // Horizontal Road (Lanes 2 & 4)
    ctx.fillRect(0, 240, 600, 120);

    // Intersection Box
    ctx.fillStyle = '#243044';
    ctx.fillRect(240, 240, 120, 120);

    // 3. Road Markings & Dashed Lines
    ctx.strokeStyle = '#e2e8f0';
    ctx.lineWidth = 2;
    ctx.setLineDash([8, 8]);

    // Center divider lines
    ctx.beginPath();
```

```
  ctx.moveTo(300, 0); ctx.lineTo(300, 240);
  ctx.moveTo(300, 360); ctx.lineTo(300, 600);
  ctx.moveTo(0, 300); ctx.lineTo(240, 300);
  ctx.moveTo(360, 300); ctx.lineTo(600, 300);
  ctx.stroke();
  ctx.setLineDash([]); // reset

  // Stop Lines (Solid White)
  ctx.lineWidth = 4;
  ctx.strokeStyle = '#ffffff';
  ctx.beginPath();
  ctx.moveTo(240, 235); ctx.lineTo(300, 235); // Lane 1 stop line
  ctx.moveTo(365, 240); ctx.lineTo(365, 300); // Lane 2 stop line
  ctx.moveTo(300, 365); ctx.lineTo(360, 365); // Lane 3 stop line
  ctx.moveTo(235, 300); ctx.lineTo(235, 360); // Lane 4 stop line
  ctx.stroke();

  // 4. Update and Draw Cars
  for (let lane = 1; lane <= 4; lane++) {
    const laneCars = cars.filter(c => c.lane === lane);
    for (let i = 0; i < laneCars.length; i++) {
      const leadingCar = (i > 0) ? laneCars[i - 1] : null;
      laneCars[i].update(leadingCar);
      laneCars[i].draw(ctx);
    }
  }

  // Filter out of bounds
  for (let i = cars.length - 1; i >= 0; i--) {
    if (cars[i].isOutOfBounds()) cars.splice(i, 1);
  }

  // 5. Draw 4 Traffic Light Posts
  drawTrafficLightPost(ctx, 215, 210, state.lanes[1].light, 'N');
  drawTrafficLightPost(ctx, 385, 215, state.lanes[2].light, 'E');
  drawTrafficLightPost(ctx, 385, 390, state.lanes[3].light, 'S');
  drawTrafficLightPost(ctx, 215, 385, state.lanes[4].light, 'W');

  requestAnimationFrame(drawIntersection);
}

function drawTrafficLightPost(ctx, x, y, lightState, orientation) {
  ctx.save();
  ctx.translate(x, y);

  // Housing box
  ctx.fillStyle = '#020617';
  ctx.strokeStyle = '#475569';
  ctx.lineWidth = 1.5;
  ctx.beginPath();
  ctx.roundRect(-10, -22, 20, 44, 4);
  ctx.fill();
  ctx.stroke();

  // Bulbs (Red, Yellow, Green)
  const redOn = (lightState === 'RED');
  const yellowOn = (lightState === 'YELLOW');
  const greenOn = (lightState === 'GREEN');

  // Red Bulb
  ctx.fillStyle = redOn ? '#ef4444' : '#450a0a';
  ctx.shadowColor = redOn ? '#ef4444' : 'transparent';
```

```
    ctx.shadowBlur = redOn ? 10 : 0;
    ctx.beginPath();
    ctx.arc(0, -12, 5, 0, Math.PI * 2);
    ctx.fill();

    // Yellow Bulb
    ctx.fillStyle = yellowOn ? '#facc15' : '#422006';
    ctx.shadowColor = yellowOn ? '#facc15' : 'transparent';
    ctx.shadowBlur = yellowOn ? 10 : 0;
    ctx.beginPath();
    ctx.arc(0, 0, 5, 0, Math.PI * 2);
    ctx.fill();

    // Green Bulb
    ctx.fillStyle = greenOn ? '#22c55e' : '#052e16';
    ctx.shadowColor = greenOn ? '#22c55e' : 'transparent';
    ctx.shadowBlur = greenOn ? 12 : 0;
    ctx.beginPath();
    ctx.arc(0, 12, 5, 0, Math.PI * 2);
    ctx.fill();

    ctx.restore();
  }

  // ============================================================================
  // 5. EXTERNAL API EXPOSURE FOR UI ACTIONS
  // ============================================================================
  window.TrafficApp = {
    triggerEmergency: function (laneId) {
      state.emergencyActive = true;
      state.emergencyLane = laneId;
      state.lanes[laneId].vehicles = Math.max(state.lanes[laneId].vehicles, 12);
      publishMqtt(`${config.topicPrefix}/control/emergency`, {
        action: 'OVERRIDE',
        lane: laneId,
        source: 'WebDashboard',
        timestamp: Date.now()
      });
      logToConsole(`[■ EMERGENCY] Ambulance dispatched to Lane ${laneId}!`, 'error');
      updateUI();
    },

    clearEmergency: function () {
      state.emergencyActive = false;
      state.emergencyLane = 0;
      publishMqtt(`${config.topicPrefix}/control/emergency`, {
        action: 'CLEAR',
        lane: 0,
        source: 'WebDashboard',
        timestamp: Date.now()
      });
      logToConsole('[✓] Emergency Cleared. Normal cycle restored.', 'success');
      updateUI();
    },

    forceLaneGreen: function (laneId) {
      state.activeLane = laneId;
      state.phase = 'GREEN';
      state.remainingSec = 20;
      state.totalPhaseSec = 20;
      for (let i = 1; i <= 4; i++) {
        state.lanes[i].light = (i === laneId) ? 'GREEN' : 'RED';
```

```
    }
    publishMqtt(`${config.topicPrefix}/control/manual`, {
      action: 'FORCE_LANE',
      lane: laneId
    });
    logToConsole(`Manual Action: Forced Lane ${laneId} to GREEN`, 'info');
    updateUI();
  },

  forceAllRed: function () {
    state.phase = 'RED';
    for (let i = 1; i <= 4; i++) {
      state.lanes[i].light = 'RED';
    }
    publishMqtt(`${config.topicPrefix}/control/manual`, {
      action: 'ALL_RED'
    });
    logToConsole('Manual Action: Intersection locked in ALL RED', 'error');
    updateUI();
  },

  resumeAuto: function () {
    state.localSimulationActive = true;
    publishMqtt(`${config.topicPrefix}/control/manual`, {
      action: 'RESUME_AUTO'
    });
    logToConsole('Manual Action: Resumed Automatic Adaptive Cycle', 'success');
    updateUI();
  },

  applyPreset: function (preset) {
    if (preset === 'morning') {
      state.lanes[1].vehicles = 22; state.lanes[1].distance_cm = 10;
      state.lanes[2].vehicles = 4;  state.lanes[2].distance_cm = 75;
      state.lanes[3].vehicles = 18; state.lanes[3].distance_cm = 15;
      state.lanes[4].vehicles = 3;  state.lanes[4].distance_cm = 80;
      logToConsole('Preset applied: Morning Commute Rush (Heavy North-South)', 'info');
    } else if (preset === 'evening') {
      state.lanes[1].vehicles = 5;  state.lanes[1].distance_cm = 70;
      state.lanes[2].vehicles = 25; state.lanes[2].distance_cm = 8;
      state.lanes[3].vehicles = 6;  state.lanes[3].distance_cm = 65;
      state.lanes[4].vehicles = 20; state.lanes[4].distance_cm = 12;
      logToConsole('Preset applied: Evening Commute Rush (Heavy East-West)', 'info');
    } else if (preset === 'lull') {
      for (let i = 1; i <= 4; i++) {
        state.lanes[i].vehicles = 1;
        state.lanes[i].distance_cm = 95;
      }
      logToConsole('Preset applied: Midnight Lull (Sparse Traffic)', 'info');
    }

    // Publish new density to broker
    for (let i = 1; i <= 4; i++) {
      publishMqtt(`${config.topicPrefix}/lane${i}/density`, {
        lane: i,
        vehicle_count: state.lanes[i].vehicles,
        distance_cm: state.lanes[i].distance_cm,
        source: 'WebDashboard_Preset'
      });
    }
    updateUI();
  }
```

```
  };

  // Event Listeners
  document.getElementById('btnSwitchModeSynthetic').addEventListener('click', () => setSystemMode('SYNTHETIC'));
  document.getElementById('btnSwitchModeReal').addEventListener('click', () => setSystemMode('REAL_ESP32'));
  document.getElementById('btnBannerClear').addEventListener('click', () => window.TrafficApp.clearEmergency());
  document.getElementById('btnClearLogs').addEventListener('click', () => {
    elMqttLogContainer.innerHTML = '<div class="text-slate-500 italic">[Logs cleared]</div>';
  });

  // Settings modal
  const modal = document.getElementById('settingsModal');
  document.getElementById('btnOpenSettings').addEventListener('click', () => modal.classList.remove('hidden'));
  document.getElementById('btnCloseSettings').addEventListener('click', () => modal.classList.add('hidden'));

  document.getElementById('btnSaveSettings').addEventListener('click', () => {
    config.brokerHost = document.getElementById('cfgBrokerHost').value.trim();
    config.brokerPort = parseInt(document.getElementById('cfgBrokerPort').value.trim(), 10);
    config.brokerPath = document.getElementById('cfgBrokerPath').value.trim();
    config.topicPrefix = document.getElementById('cfgTopicPrefix').value.trim();

    localStorage.setItem('cfg_broker_host', config.brokerHost);
    localStorage.setItem('cfg_broker_port', config.brokerPort);
    localStorage.setItem('cfg_broker_path', config.brokerPath);
    localStorage.setItem('cfg_topic_prefix', config.topicPrefix);

    elTopicBaseDisplay.textContent = `Topic: ${config.topicPrefix}/#`;
    modal.classList.add('hidden');

    if (mqttClient) mqttClient.end(true);
    initMqtt();
  });

  // Update GitHub link in navbar if hosted on GitHub Pages
  if (window.location.hostname.includes('github.io')) {
    const parts = window.location.pathname.split('/').filter(Boolean);
    if (parts.length > 0) {
      const repoName = parts[0];
      const user = window.location.hostname.split('.')[0];
      document.getElementById('repoLink').href = `https://github.com/${user}/${repoName}`;
    }
  }

  // Initialize
  elTopicBaseDisplay.textContent = `Topic: ${config.topicPrefix}/#`;
  initMqtt();
  updateUI();
  requestAnimationFrame(drawIntersection);

})();
```

Code Listing: 8. Dashboard Styling & Animations

Path: css/style.css | **Type:** CSS3

```
/* Custom dashboard styles and animations */

/* Smooth custom scrollbars */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}
::-webkit-scrollbar-track {
  background: #020617;
}
::-webkit-scrollbar-thumb {
  background: #1e293b;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #334155;
}

/* Pulsing effect for green light active */
@keyframes glow-green {
  0%, 100% {
    box-shadow: 0 0 15px rgba(34, 197, 94, 0.4);
  }
  50% {
    box-shadow: 0 0 25px rgba(34, 197, 94, 0.8);
  }
}

.active-lane-glow {
  border-color: rgba(34, 197, 94, 0.5) !important;
  animation: glow-green 2s infinite ease-in-out;
}

/* Emergency siren flash */
@keyframes emergency-glow {
  0%, 100% {
    background-color: rgba(225, 29, 72, 0.15);
  }
  50% {
    background-color: rgba(225, 29, 72, 0.35);
  }
}

.emergency-flash {
  animation: emergency-glow 1s infinite alternate;
}

/* High-DPI Canvas Rendering */
#intersectionCanvas {
  image-rendering: -webkit-optimize-contrast;
  image-rendering: crisp-edges;
}
```

Code Listing: 9. GitHub Actions CI/CD Deployment

Path: .github/workflows/deploy.yml | **Type:** YAML Workflow

```yaml
name: Deploy Dashboard to GitHub Pages

on:
  push:
    branches:
      - master
      - main
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: false

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Upload Artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```